

Institut für Maschinelle Sprachverarbeitung
Universität Stuttgart
Pfaffenwaldring 5B
D-70569 Stuttgart

**From speech to image;
a novel approach to understand the
hidden layer mechanisms of deep
neural networks in automatic speech
recognition**

Mahbub Ul Alam
Master thesis

Prüfer: Dr. Antje Schweitzer
Junior-Prof. Dr. Ngoc Thang Vu
Betreuer: Junior-Prof. Dr. Ngoc Thang Vu

Start Date: 15.05.2017
End Date: 26.10.2017

Erklärung (Statement of Authorship)

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbständig verfasst habe und dabei keine andere als die angegebene Literatur verwendet habe. Alle Zitate und sinngemäßen Entlehnungen sind als solche unter genauer Angabe der Quelle gekennzeichnet. Die eingereichte Arbeit ist weder vollständig noch in wesentlichen Teilen Gegenstand eines anderen Prüfungsverfahrens gewesen. Sie ist weder vollständig noch in Teilen bereits veröffentlicht. Die beigefügte elektronische Version stimmt mit dem Druckexemplar überein.¹

(Mahbub Ul Alam)

¹Non-binding translation for convenience: This text is the result of my own work, and any material from published or unpublished work of others which is used either verbatim or indirectly in the text is credited to the author including details about the exact source in the text. This work has not been part of any other previous examination, neither completely nor in parts. It has neither completely nor partially been published before. The submitted electronic version is identical to this print version.

”...One thing at least is certain - This Life flies;
One thing is certain and the rest is Lies -
The Flower that once has blown forever dies.”
- Rubaiyat

Acknowledgements

At first I would like to thank my thesis supervisor Dr. Ngoc Thang Vu, Assistant Professor, Institut für Maschinelle Sprachverarbeitung at Universität Stuttgart. I am forever grateful for his extraordinary kindness, and invaluable suggestions regarding this thesis.

I would like to thank my beloved parents, sister, and brother for their selfless support and unconditional love. I would like to thank all of my friends, relatives, and colleagues, who have helped me to shape my thinking process, and helped me to be the person I am now.

Finally, I am expressing my gratitude to Tanzia Haque Tanzi, my loving wife. Without you, this work would not have been possible.

Let all of us prosper together.

Abstract

Currently deep neural network based models are showing state of the art results in automatic speech recognition domain. However, still plenty of questions are left undetermined to understand the computational data flow, and feature classification process of hidden layers in deep neural networks. Visualization in hidden layer mechanisms can provide valuable insights regarding this understanding. In order to address this issue, this work reviews the recent visualization techniques used in computer vision domain for kernels and deep neural networks, and then applies one of these techniques ‘layer-wise relevance propagation’ algorithm in automatic speech recognition task for the first time. This work attempts to make the hidden layers in deep learning systems more interpretable by adding another visualization tool in the whole automatic speech recognition process chain. Several hidden layer mechanisms such as ‘learning after each epoch change’, ‘effectiveness of context-window’, ‘impact of speech quality degradation in speech recognition’, and ‘difference between identifying vowels and consonant phones’ are analyzed using relevance score based images.

Index Terms: deep neural networks, deep learning, visualization, automatic speech recognition, natural language processing

Contents

1	Introduction	7
2	Related Work	9
2.1	Kernel Based Works	9
2.2	Deep Learning Based Works	9
2.2.1	Deconvolution Method	10
2.2.2	Sensitivity Analysis	11
2.2.3	Layer-wise Relevance Propagation (LRP)	11
2.3	My Contribution	11
3	Background	13
3.1	Automatic Speech Recognition	13
3.2	Deep Learning	15
3.2.1	Multilayer Perceptron Networks	18
3.3	Heatmaps And LRP Methods	21
3.3.1	Simple LRP	24
3.3.2	Flat LRP	26
3.3.3	WW LRP	26
3.3.4	Epsilon LRP	27
3.3.5	Alphabeta LRP	28
4	Proposed Framework: ‘deep-speech-vis’	29
4.1	Input Module	30
4.2	Feature Extraction Module	30

4.3	Neural Network Module	31
4.4	Visualization Module	31
4.5	Kaldi Processing Module	33
5	Resources	34
5.1	Automatic Speech Recognition Toolkit: Kaldi	34
5.2	Speech Quality Degradation Toolkit: Acoustic Simulator	37
5.3	Deep Learning Library: Tensorflow	40
5.4	Data Set: The Wall Street Journal Corpus	42
6	Experiments And Results	43
6.1	Kaldi Post-processing	44
6.2	Neural Network Implementation	45
6.3	Experimental Setup	46
6.3.1	Experiment 1: Epoch Change Visualization	47
6.3.2	Experiment 2: Context-Window Visualization	47
6.3.3	Experiment 3: Noisy Data Decoding Visualization	48
6.3.4	Experiment 4: Vowels And Consonants Visualization	48
6.4	Heatmaps And Discussion	49
6.4.1	Experiment 1	50
6.4.2	Experiment 2	52
6.4.3	Experiment 3	55
6.4.4	Experiment 4	57
7	Conclusion	67
8	Future Work	68

Appendix I: Acronyms	69
Appendix II: Mathematical Reference	72

1 Introduction

Recently, different deep learning techniques using deep neural networks (DNNs) are showing impressive results in automatic speech recognition (ASR) domain [Hinton et al. (2012)]. Deep learning technique uses the representation of high level abstractions in data using deep graphs with multiple processing layers, composed of multiple linear and non-linear transformations.

There is a notion in the research community that, DNNs are black-box incomprehensible models, as the non-linearity creates an opaqueness in interpretability for both intuitive and quantitative cases. We should address this issue for various crucial reasons. Recently machine learning and specially deep learning algorithms are reaching every corner of society with amazing success in different applications [Du et al. (2016)]. We should consider interpretability as a gateway between research community and general society for mutual convenience, and aversion of any possible negative rumor related to artificial intelligence in the media. In this way, common people will understand clearly about the inner mechanisms about different artificial intelligence (AI) based applications. They will feel secure, and will use the complex models for different application more easily and convincingly, without having any fear of the unknown. This will ensure the future progress in this emerging field. Businessmen and policymakers will accept the prediction and calculation, and can utilize these output much more convincingly for various important sectors including banking, planning, and economic policy making. We should also respect the human decision making process by assigning different degrees of responsibility in the implementation and utilization of deep neural networks. Interpretability can be an effective tool for this scenario. In many countries customers and users have legal rights to have the full explanation on their purchased products. Therefore, to avoid legal issues in the business of deep learning technique based applications, and commodities, interpretability is quite essential. From the engineering perspective it is also quite important. From optimizing and scaling the system, to troubleshooting

the model, interpretability is always essential. Moreover, interpretability can help us to gain new insights about the existing problem.

It is therefore evident that, to address these issues we have to start from the very beginning of implementing the deep neural networks, with an additional visualization aspect along with the current existing training and decoding parts. Simply put, we want to visualize the inner mechanisms of both the training and decoding processes of the deep neural network based models. Although extensive research works have been done in this regard in computer vision domain, in ASR domain this is still quite unexplored.

In this regard, my aim is to work on different visualization techniques in ASR domain. My intention is to observe the actual data flow or recombination mechanism, that takes place inside, when we are using deep neural networks for automatic speech recognition, in the form of activated neuron’s heatmap image. In my thesis work, I used the layer-wise relevance propagation (LRP) algorithm [Bach et al. (2015)] to address this issue. I proposed a novel framework for my thesis work named ‘deep-speech-vis’ where the whole neural network training, decoding, and visualization process can be completed as a sequential pipe-lined way. Using this framework, I successfully observed the neural network, by creating heatmaps, and performed analysis regarding the performance improvement after each iteration (epoch change), effectiveness of context-window, impact of speech quality degradation in input data, and the different relevancy on detecting vowel and consonant phones. To my knowledge this is the first approach to apply computer vision based LRP visualization techniques in automatic speech recognition domain.

The thesis chapters are organized as follows: in section 2, I have discussed about the related works in this visualization topic. I emphasized on three particular visualization methods, (sensitivity analysis, deconvolution, and layerwise relevance propagation) and discussed about them individually. In section 3, I discussed about automatic speech recognition system, deep neural network architecture and relevance heatmap calculation mechanism extensively. In section 4, I presented my own framework ‘deep-speech-vis’

with full details. In section 5, I discussed about the necessary toolkits, programming libraries, simulators, and data set that I have used for my thesis work. Section 6 describes the different experiments setups and their results, and heatmaps. I discussed about the insights and findings related to these results here. In section 7, I tried to provide a conclusion of my work. Finally, in section 8, I discussed the possibilities of future work.

2 Related Work

The visualization and interpretation of non-linearity is a recent research topic. Here I will briefly discuss about the important works, categorized as different kernel based works and deep learning based works.

2.1 Kernel Based Works

Braun et al. (2008) proposed an algorithm which visualizes dimensionality reduction, by restoring principal component analysis (PCA) items of the chief kernel. This was later used to identify the effectiveness of classification. Baehrens et al. (2010) proposed local explanation vectors to explain locality in classification for individual data point. They viewed local explanation vectors as class probability gradients. Hansen et al. (2011) proposed a kernel-based method to evaluate the reliability of relevant molecular features.

2.2 Deep Learning Based Works

After the overwhelming superiority of neural networks in different classification tasks, the focus also shifted on how a DNN actually learns, or how to visualize and interpret a single neuron, or several neurons containing hidden layers. Krizhevsky et al. (2012) showed the effectiveness of convolutional neural network (CNN) with rectified linear unit (ReLU) for image classification, and visualized the learning process in convolutional kernels. Zeiler and Fergus

(2014) proposed a deconvolution method to detect patterns in input image while classification. Mahendran and Vedaldi (2015) tried to reconstruct the image from the invert representations in convolutional layers. They observed that, photographically reliable information with the variance in geometric and photometric degree can be gathered from convolutional layers. Szegedy et al. (2013) observed that, the difference is almost non-existent between individual high level neurons and random linear combinations of high level neurons. They also showed that, it is possible to generate wrong classification in different networks, by using the same hardly perceptible perturbation, by maximizing the network’s prediction error. Goodfellow et al. (2014) showed that this wrong classification process is due to neural networks linear nature. Yosinski et al. (2015) proposed two tools for visualizing images and videos. First tool plots interactively layerwise activations, after the training on a new test image or video. The second one improves it by computing the activations in single neuron level. Dosovitskiy and Brox (2015) showed a new approach with the integration of an up-convolutional neural network.

Now I will emphasize on the works where visualization is performed to identify the impact of the specific areas of input data (in these related works the input data are image data), which is used for classification task. These works can be classified into three major subcategories and I will describe these categories here briefly.

2.2.1 Deconvolution Method

To generate interpretable patterns or heatmap images, Zeiler and Fergus (2014) utilized the weights of each layer in a backward driven approach from output layer towards the input layer. The main limitation of this approach is, it is intuitive without a quantifiable measurement aspect, and can only work in the max pooling layers of convolutional neural networks.

2.2.2 Sensitivity Analysis

Sensitivity analysis focuses on the local scores on partial derivatives at the provided instance [Baehrens et al. (2010), Rasmussen et al. (2012), Simonyan et al. (2013)]. Simonyan et al. (2013) used this technique to visualize the input image sensitivity in DNNs. It is important to note that, partial derivatives do not explain the changing phenomenon of a classifier’s decision. It only provides us the idea about what makes a particular instance more or less worthy for a particular class category.

2.2.3 Layer-wise Relevance Propagation (LRP)

Bach et al. (2015) proposed this approach, which is applicable for different neural networks rather than only convolutional neural networks. It tries to provide the difference of a classification function $f(x)$, relative to the neural state $f(x) = 0$. It uses a conservation principle for back propagation without using gradients. It also guarantees that all the output data are fully propagated backwards without losing the positivity or negativity of the evidence. The different variations of this approach will be elaborately discussed in section 3.3, as I have used this approach for the computational work of my thesis.

2.3 My Contribution

I used LRP techniques for my task, as it can be used for different neural networks and activation functions. To my knowledge, this is the first instance of using this technique for visualization and interpretation in ASR domain. I calculated the relevancy score for each neuron based on this technique and represented them in RGB heatmaps to differentiate in my experiments.

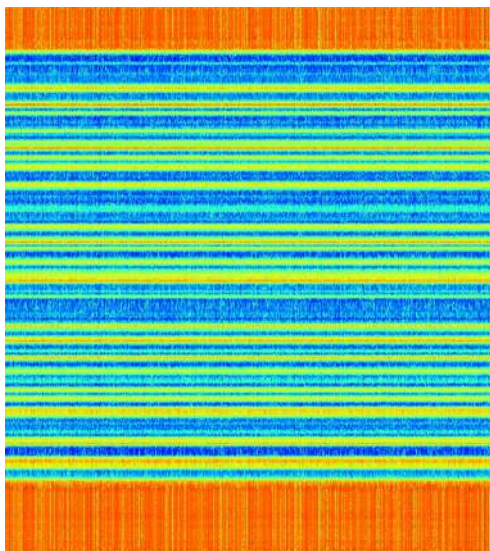


Figure 1: Relevance heatmap representation of an input speech utterance

Figure 1 is showing such a relevance heatmap of a speech utterance. Each of the 6*6 pixel areas is denoting a particular feature data point.

I have considered 40fbank feature for the input audio data as it resembles human hearing most [Rosen and Howell (2011)].

I have created a completely independent framework named as ‘deep-speech-vis’, which works after Kaldi speech toolkit [Povey et al. (2011)], and provides the training, decoding, and visualization independently.

I have observed the performance improvement after each epoch based on relevance heatmaps. The effectiveness of context-window is also shown by the heatmaps. Automatic speech recognition systems perform poorly on speech quality degraded data. I tried to identify what is actually going on inside neural networks, when we try to decode speech quality degraded data. Finally, I tried to show the different relevance area of neural networks, used for identifying different consonant and vowel phones.

3 Background

In this section, I will introduce automatic speech recognition domain and deep neural networks paradigm, and discuss about heatmap generation using LRP techniques much more elaborately.

3.1 Automatic Speech Recognition

While verbal communication is quite natural for most humans, computers have historically dealt primarily with text data. As many situations make verbal communication more desirable for humans, the ability to automatically convert spoken words to text is quite useful in the context of human-computer interaction. While this task has proved to be a difficult one, great strides have been made, and current approaches are able to convert speech to text with reasonable accuracy [Huang et al. (2001)].

Automatic speech recognition systems consist of two major components: an acoustic model (AM) and a language model (LM). The acoustic model models the statistical correlations between input audio sequences and corresponding word sequences in the target language. On the other hand, the language model is responsible for modeling the statistical properties of word sequences in that language.

Formally, the acoustic model models the conditional probability, $P(X|W)$ where X is the acoustic signal of a piece of speech in the target language, and W is a sequence of words in the same language. This acoustic model is implemented in two parts: A pronunciation dictionary and a phones model. The pronunciation dictionary maps each word or lexical unit in the target language to one or more sequences of phonemes, drawn from an inventory of phonemes specific to the language. Some words with multiple alternate pronunciations will be mapped to multiple possible phoneme sequences.

While the pronunciation dictionary maps between phoneme sequences and character sequences, the phones model maps between audio samples

and phoneme sequences. Specifically, the phones model assigns each audio signal a probability distribution over possible phoneme sequences. This is done via a hidden markov model (HMM) [Young (2008)]. The states of this hidden markov model are derived from the language’s phoneme inventory, with a state for each phoneme’s onset, middle, and offset. In addition, in a context-dependent model, even more states are used, as each phoneme may be responsible for multiple sets of states depending on its context. Each state then specifies its emission probability distribution, which is a probability distribution over 10ms audio samples. Traditional ASR model specifies these distributions using a gaussian mixture model (GMM) [Povey et al. (2010)]. The coefficients for the phone model, including the gaussian coefficients and the transition probabilities, are learned from a large corpus of annotated speech using an expectation-maximization algorithm.

The language model deals only with the statistics of text in the target language, not with the phonetic properties. Formally, the language model assigns a probability $P(W)$ to an arbitrary sequence of words W . This is done using a simple n-gram model, which is trained on a text corpus [Huang and Deng (2010)].

By Bayes’ theorem, we know that,

$$(1) \quad P(W|X) \propto P(X|W)P(W)$$

Since we can calculate both of these factors, the speech recognition task is reduced to a simple search problem of finding, or at least approximating,

$$(2) \quad \arg \max_W P(W|X) = \arg \max_W P(X|W)P(W)$$

This search problem is known as decoding in the domain of ASR. It is generally conceptualized as a graph search, over a decoding graph whose edges correspond to words, and whose paths correspond to strings.

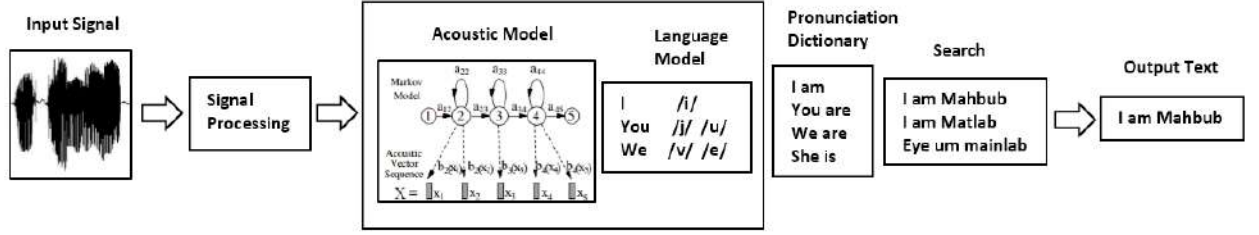


Figure 2: Automatic speech recognition system overview

3.2 Deep Learning

Deep learning is a branch of machine learning approaches for solving different artificial intelligence related problems. It has been clearly shown again and again that, deep learning performs overwhelmingly better in almost all the sections of computer vision and natural language processing domain, particularly in automatic speech recognition domain [Du et al. (2016)]. The algorithms of deep learning techniques are based on the imitation of the structure and function of the network connectivity of human brain cell ‘neurons’, and usually referred as artificial neural networks. When we are using sufficiently large models and sufficiently large data sets of labeled training examples, with many hidden layers, then we refer these artificial neural networks as deep neural networks. Figure 3 shows the position of deep learning approach among different approaches in artificial intelligence domain.

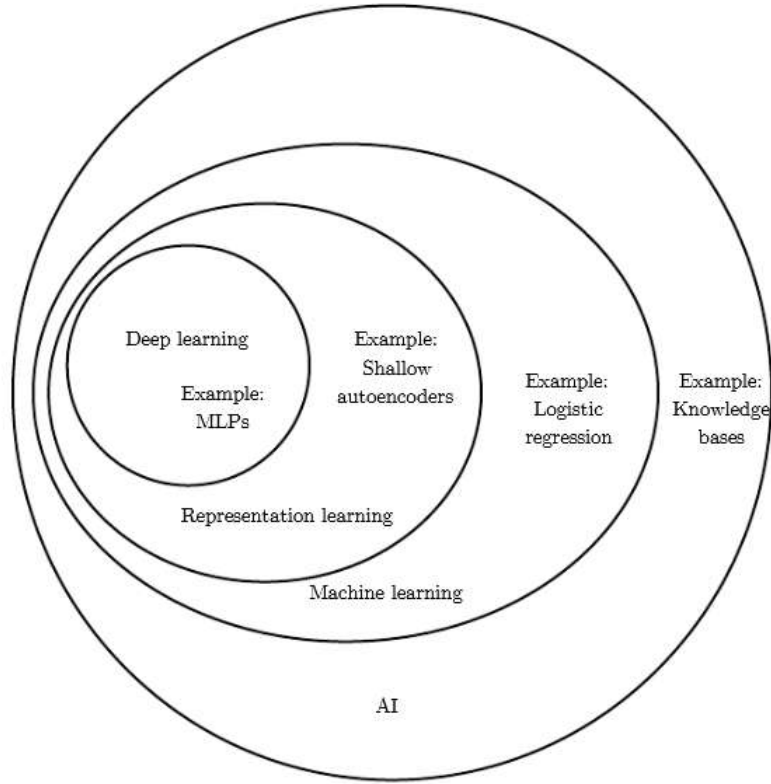


Figure 3: Position of deep learning among different approaches in artificial intelligence domain [Goodfellow et al. (2016)]

Deep learning approach learns from the input data features, by using a hierarchical representations of input features information, from lower level compositions to higher level compositions. This type of abstraction at multiple levels allows to learn features automatically using complex formation of functions, which map the input to the output straightforwardly without any need of human feature engineering. It also enables computers to follow the similar learning process of humans, by learning complex knowledge using simpler layers of basic knowledge bases. Therefore It also represents the artificial intelligence, by mimicking human learning process effectively. Figure 4 shows the conceptual hierarchy graph of deep learning comparing with other methods. We can see that, in case of deep learning approach the graph is rel-

atively ‘deep’. This is also another technical reason for labeling this approach as deep learning.

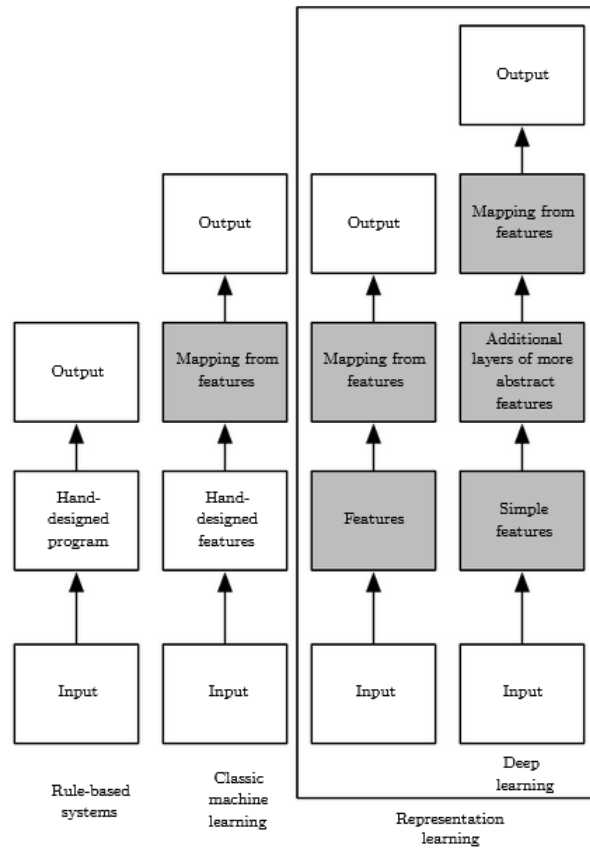


Figure 4: “Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data” [Goodfellow et al. (2016)]

Modern state of the art deep learning is focused on training deep (multi layered) neural network models using the backpropagation algorithm. The most popular techniques are:

- Multilayer perceptron networks (MLP) or feed-forward neural networks
- Convolutional neural networks (CNN)
- Long short-term memory recurrent neural networks (LSTM)

3.2.1 Multilayer Perceptron Networks

Multilayer perceptron (MLP) networks are the most recognized examples of deep learning based models. It is also known as feed-forward neural networks. The main aim of this architecture is to approximate a function $f^*(\mathbf{x})$, which maps input $\mathbf{x}$ to a classification label y where $y = f^*(\mathbf{x})$. We describe this mapping $\mathbf{y} = f(\mathbf{x}; \theta)$ in our feed-forward network, and then it learns the most appropriate values of the parameter θ , so that we can achieve the best result for the function approximation.

We are referring this network as feed-forward because, the information-flow (from input $\mathbf{x}$ through different computations to get the approximation function and $\mathbf{y}$) is unidirectional from input $\mathbf{x}$ to output $\mathbf{y}$. This model can not feedback the output to it's inputs. We also refer it as multilayer perceptron (MLP) because, it contains one or more hidden layers along with input and output layers (the definition of layer is described below), and initially the simple architecture without hidden layer was termed as perceptron [Rosenblatt (1958)] .

The term 'network' is used because, in this approach we are utilizing several individual functions composed together. We can view this composition as a directed acyclic graph. In this way, we can describe our approximation function as a connection of function chains, $f(\mathbf{x}) = f^{(3)}(f^{(2)}(f^{(1)}(\mathbf{x})))$. In deep learning terminology, $f^{(1)}$ can be referred as first layer of the network, $f^{(2)}$ as second layer and so on. The 'depth' of the model can be determined by the total length of this chain. The longer the chain, the deeper the network (practical representation of 'deep learning'). We term the final layer as output layer. When we train our neural network, our aim is to generate $f(\mathbf{x})$, so that it can be closer to $f^*(\mathbf{x})$. We obtain many noisy examples of $f^*(\mathbf{x})$ from the training data, where each example is provided with a label $y \approx f^*(\mathbf{x})$. The layer where the $\mathbf{x}$ values are provided is called input layer. Apart from the input and output layer, other layers are not directly specified, when we design the feed-forward network. It is the responsibility of learning algorithms to

determine the formation of these layers, and to get the best approximation of $f^*(\mathbf{x})$. Therefore, we can say that, training data does not reveal us the mechanism or result of these layers. This is why, these layers are referred as hidden layers.

The term ‘neural’ indicates that, this model is loosely based on the neuroscience mechanisms of human brain. Usually each hidden layer is vector valued, where the activity of each element of this vector can be viewed like the function of neuron in human brain. These elements or units work in parallel, and each represents a vector to scalar function. We are comparing these with neurons because, like neurons, these unit elements are also receiving various input information from different units, and based on these inputs, they compute outputs using different activation functions, and provide these outputs to other units for further computations.

There are several activation functions to compute the hidden layer values. The most important ones are discussed here,

- **Sigmoid:** It takes a real-valued input and squashes it to range between 0 and 1

$$(3) \quad \sigma(x) = \frac{1}{(1 + \exp(-x))}$$

If sum of all the outputs is equal to one, then we call it as soft-max function. We can consider these outputs as probabilities, and use soft-max function for multi-label classification. Therefore, soft-max function is usually used with the output layer, for probabilistic classification.

- **tanh:** It takes a real-valued input and squashes it to the range $[-1, 1]$

$$(4) \quad \tanh(x) = 2\sigma(2x) - 1$$

- **ReLU:** ReLU stands for Rectified Linear Unit. It takes a real-valued input and thresholds it at zero (replaces negative values with zero)

$$(5) \quad f(x) = \max(0, x)$$

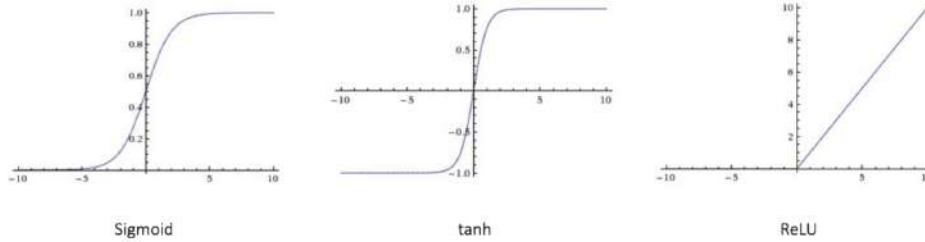


Figure 5: Different activation functions

Figure 5 shows the graphical representation of these activation functions.

We need to compute the gradients of different functions, in order to make deep neural networks learn the best parameters. Feed-forward neural network takes an input, and produces an output through propagation of all the units of each hidden layers, as mentioned before. We call this forward propagation. In the training time, this forward propagation finally provides us a scalar cost $\mathbf{J}(\theta)$. We then use back-propagation algorithm [Rumelhart et al. (1988)] to flow this information from cost $\mathbf{J}(\theta)$ backward to the network, in order to calculate and obtain the gradient values. We then use another learning algorithm (ex. stochastic gradient descent [Kiefer et al. (1952)]) to learn the best parameters, so that it provides the best approximations.

We use several iterations (also known as epochs) of the whole training data, to complete the training procedure. When this training is finished, we test our network with new inputs, and evaluate the predictions. If the prediction is satisfactory, then we have an effective model for our desired task (ex. automatic speech recognition).

The most important part while designing this network is, the number of hidden layers, the number of neurons in each layer, the choice of activation function and the choice of learning algorithm. These parameters value varies a lot depending on the task and input data.

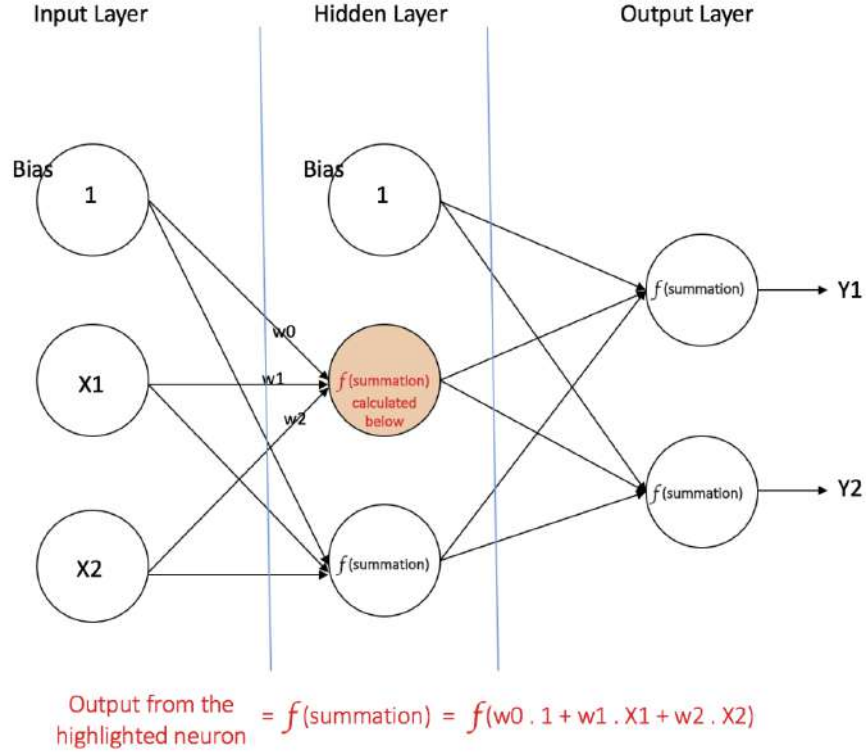


Figure 6: Multi layer perceptron networks

Figure 6 shows a simple feed-forward neural network architecture with three neurons in one hidden layer. Here f refers to activation function, and weight (w) and bias (b) are parameters to tune for learning, using back-propagation and learning algorithms.

3.3 Heatmaps And LRP Methods

Let $f : \mathbb{R}^d \rightarrow \mathbb{R}^+$ where $\mathbf{x} \in \mathbb{R}^d$ is the input. Here in this context we are considering the input data as image data. Now we will decompose this image as $\mathbf{x} = x_p$ where p is a particular pixel. $f(\mathbf{x})$ denotes the presence of certain pattern or object in the image. Therefore $f(\mathbf{x}) = 0$ means, there is no such object or pattern. Now relevance score $R_p(\mathbf{x})$ indicates the extent of contribution of pixel p to interpret the classification decision $f(\mathbf{x})$, for

the whole image $\mathbf{x}$. So we can keep all the relevance score in a heatmap as $\mathbf{R}(\mathbf{x}) = R_p(\mathbf{x})$. This will be the same dimension as input image, and also can be shown as another image [Montavon et al. (2017)].

In practice Montavon et al. (2017) proposed following certain properties to be satisfied for a heatmap,

“ **Definition 1.** A heatmapping $\mathbf{R}(\mathbf{x})$ is conservative if the sum of assigned relevances in the pixel space corresponds to the total relevance detected by the model, that is,

$$(6) \quad \forall \mathbf{x} : f(\mathbf{x}) = \sum_p R_p(\mathbf{x})$$

Definition 2. A heatmapping $\mathbf{R}(\mathbf{x})$ is positive if all values forming the heatmap are greater or equal to zero, that is:

$$(7) \quad \forall \mathbf{x}, p : f(\mathbf{x}) = R_p(\mathbf{x}) \geq 0$$

”[Montavon et al. (2017)]

Now considering these constraints, we can apply taylor decomposition to generate these heatmaps. We can use taylor expansion of function f at a significant root point $\tilde{\mathbf{x}}$, such that $f(\tilde{\mathbf{x}}) = 0$. In such scenario, the taylor expansion series will be as follows,

$$(8) \quad \begin{aligned} f(\mathbf{x}) &= f(\tilde{\mathbf{x}}) + \left(\frac{\partial f}{\partial \mathbf{x}} \bigg|_{\mathbf{x}=\tilde{\mathbf{x}}} \right)^T (\mathbf{x} - \tilde{\mathbf{x}}) + \varepsilon \\ &= 0 + \sum_p \frac{\partial f}{\partial x_p} \bigg|_{\mathbf{x}=\tilde{\mathbf{x}}} (x_p - \tilde{x}_p) + \varepsilon \end{aligned}$$

Here sum operation (based on combining the pixels $\{\tilde{x}_p\}$) denotes the pixel values of root point $\tilde{\mathbf{x}}$. This sum can be recognized as relevance score

$R_p(\mathbf{x})$ for all image pixels. For second or more orders terms, we are using ε here, which is computationally expensive to compute. Therefore we are only considering the first order terms here. In this scope, heatmap can be identified as the element wise production,

$$(9) \quad \mathbf{R} = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\mathbf{x}=\tilde{\mathbf{x}}} \odot (\mathbf{x} - \tilde{\mathbf{x}})$$

Now for the deep neural network, we will use this same concept in a clever way. This deep taylor decomposition method mimics the traditional divide-and-conquer approach. It uses the idea that, basically a deep neural network can be subdivided into different subfunctions of adjacent layers. So instead of considering function f as a whole, we can map it as a set of neurons $\{x_i\}$ in a particular layer to the relevance R_j assigned to a neuron x_j in the next layer. Let consider that, these two objects have some relations with the aspect to the function $R_j(\{x_i\})$, then we can apply taylor decomposition on this local function to redistribute relevance R_j to lower layer relevances $\{R_i\}$. Therefore the taylor decomposition of $\sum_j R_j$ at $\{x_i\}$ represents,

$$(10) \quad \begin{aligned} \sum_j R_j &= \left(\left. \frac{\partial(\sum_j R_j)}{\partial \{x_i\}} \right|_{\{\tilde{x}_i\}} \right)^T (\{x_i\} - \{\tilde{x}_i\}) + \varepsilon \\ &= \sum_i \sum_j \left. \frac{\partial R_j}{\partial x_i} \right|_{\{\tilde{x}_i\}} (x_i - \tilde{x}_i) + \varepsilon \end{aligned}$$

Here $\sum_j \left. \frac{\partial R_j}{\partial x_i} \right|_{\{\tilde{x}_i\}} (x_i - \tilde{x}_i)$ represents R_i .

Equation 10 is redistributing the relevance from one layer to the below layers. ε represents the higher order taylor values. $\left|_{\{\tilde{x}_i\}}$ confirms that the derivative is calculated on a particular root point $\{\tilde{x}_i\}$, where $\sum_j$ approaches on all the neurones of a particular layer, and $\sum_i$ moves on all the neurons in

lower layer. Therefore this equations satisfies to successfully identify the relevance of lower layer, and if we apply the same taylor decomposition technique over all the lower layers, then we can evaluate the whole heatmap (divide and conquer approach).

To sum it up, after following the constraints and applying deep taylor decomposition [Montavon et al. (2017)], we can generate heatmaps for different LRP variants.

3.3.1 Simple LRP

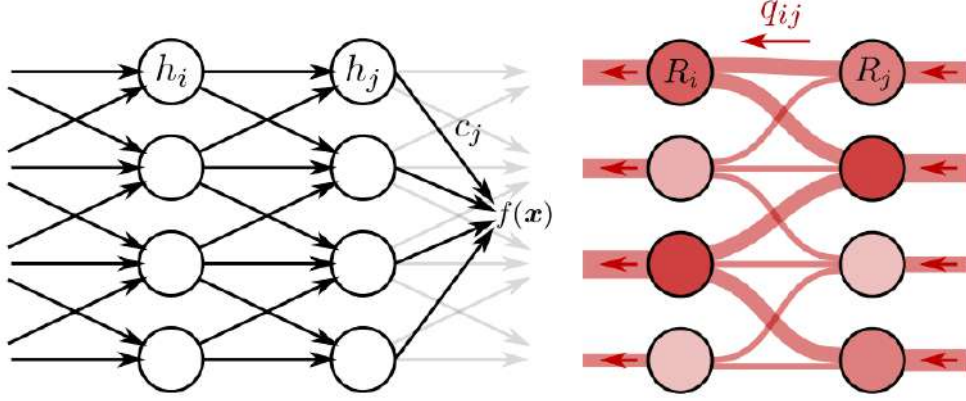


Figure 7: Simple Layerwise Relevance Propagation

Let's take a simple example shown in figure 7, where we want to get the relevance score from output $f(\mathbf{x})$. Here $c_j > 0$ is locally constant. So we can decompose it using deep taylor decomposition [Montavon et al. (2017)] as, $f(\mathbf{x}) = \sum_j R_j$ with, $R_j = h_j c_j$.

Now we can propagate it backwards using the same criterion, so it becomes, $f(\mathbf{x}) = \sum_i R_i$ with, $R_i = h_i c_i$ and, $c_i = \sum_j \frac{w_{ij}^+ h_j c_j}{\sum_j h_i w_{ij}^+}$.

Now the relevance score R_i can also be written as,

$$(11) \quad R_i = \sum_j \left(\frac{h_i w_{ij}^+}{\sum_j h_i w_{ij}^+} \right) R_j$$

We can explain equation 11 as a propagation stream of relevance, from backward layers to forward layers in the neural network. Here $q_{ij} = \frac{h_i w_{ij}^+}{\sum_j h_i w_{ij}^+} R_j$ is the relevance fraction from neuron j to neuron i .

In practice, relevance propagation does not require a strict deep taylor decomposition [Montavon et al. (2017)]. We can utilize any propagation function $q_{ij} = g(h_i, w_{ij}, \dots)$ with $\sum_i q_{ij} = 1$. We can also tune this function for different measurement of decomposition quality, which are described in sections 3.3.2 to 3.3.5. These variations help to utilize LRP with other models [e.g. FisherBoW (bag of words) + SVMs (support vector machines)] than neural network based models, and other activation functions than ReLU.

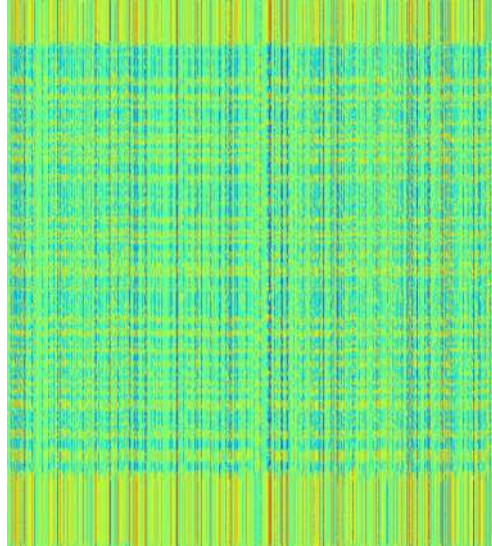


Figure 8: Heatmap from Simple LRP method

3.3.2 Flat LRP

Flat LRP distributes relevance for each output evenly to the output neurons' receptive fields. Therefore for fully connected layers, this results in a uniform lower layer relevance map.

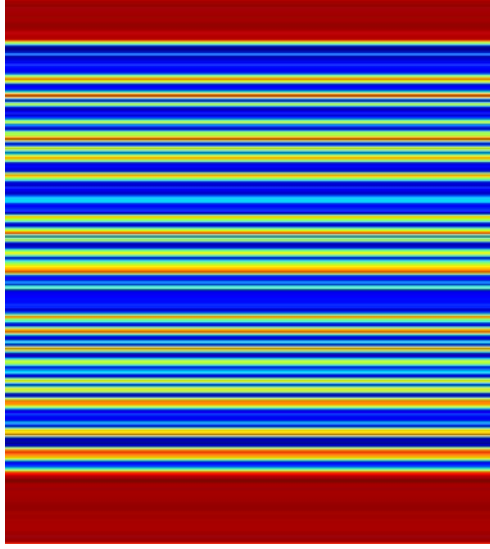


Figure 9: Heatmap from Flat LRP method

3.3.3 WW LRP

Based on the square magnitude of the weights, WW LRP redistributes relevance. It then pools relevance across all neurons j , by using the same fashion. It is worth mentioning that, it is also valid for the final layer $R_j = 0$, where the actual point x_i is already a root, and therefore relevance propagation is no longer required.

$$(12) \quad R_i = \sum_j \left(\frac{w_{ij}^2}{\sum_{i'} w_{i'j}^2} \right) R_j$$

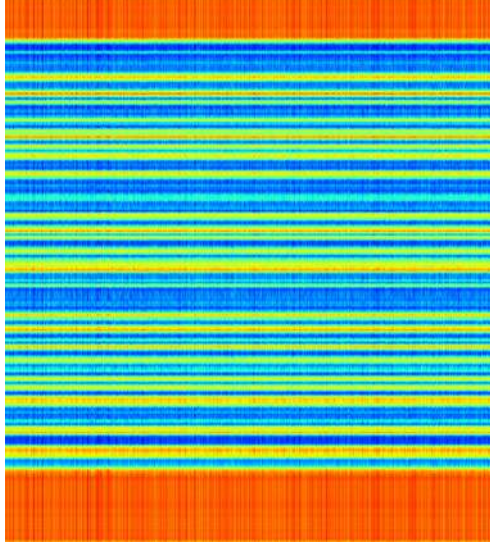


Figure 10: Heatmap from WW LRP method

3.3.4 Epsilon LRP

For a small value in simple LRP, relevance can take unbounded values. To fix this, a predefined stabilizer $\varepsilon \geq 0$ is introduced. Relevance is fully absorbed if the stabilizer ε becomes very large [Bach et al. (2015)].

$$(13) \quad R_{i \leftarrow j}^{l,l+1} = \begin{cases} \frac{z_{ij}}{z_j + \varepsilon} R_j^{l+1} & z_j \geq 0 \\ \frac{z_{ij}}{z_j - \varepsilon} R_j^{l+1} & z_j < 0 \end{cases}$$

The conservation law then becomes,

$$(14) \quad \sum_i R_{i \leftarrow j}^{l,l+1} = \begin{cases} R_j^{l+1} (1 - \frac{b_j + \varepsilon}{z_j + \varepsilon}) & z_j \geq 0 \\ R_j^{l+1} (1 - \frac{b_j - \varepsilon}{z_j - \varepsilon}) & z_j < 0 \end{cases}$$

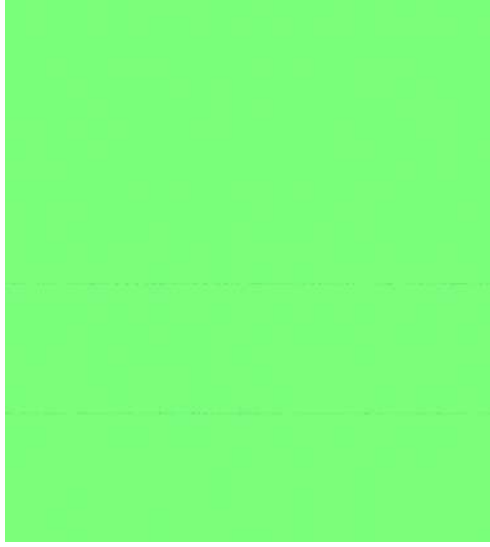


Figure 11: Heatmap from Epsilon LRP method

3.3.5 Alphabetical LRP

There is another way to solve the unbounded value problem.

“Let $z_j^+ = \sum_i z_{ij}^+ + b_j^+$ and $z_j^- = \sum_i z_{ij}^- + b_j^-$ where $-$ and $+$ denote the negative and positive part of z_{ij} and b_j . Relevance propagation is now defined as,

$$(15) \quad R_{i \leftarrow j}^{l,l+1} = R_j^{l+1} \left(\alpha \frac{z_{ij}^+}{z_j^+} + \beta \frac{z_{ij}^-}{z_j^-} \right)$$

”[Bach et al. (2015)]

We can control the importance of positive (success) and negative (fail) classification cases, by using different values of α and β .

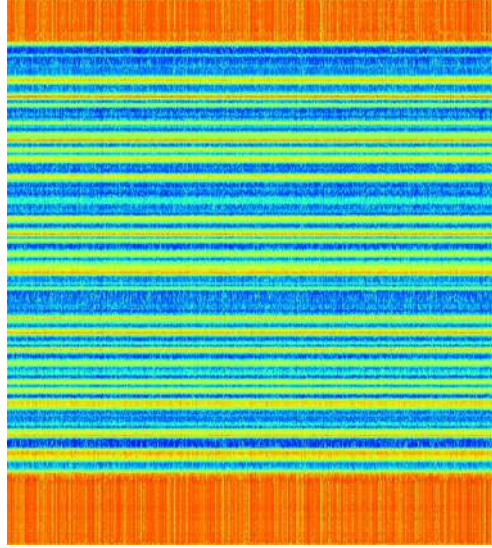


Figure 12: Heatmap from Alphabeta LRP method

4 Proposed Framework: ‘deep-speech-vis’

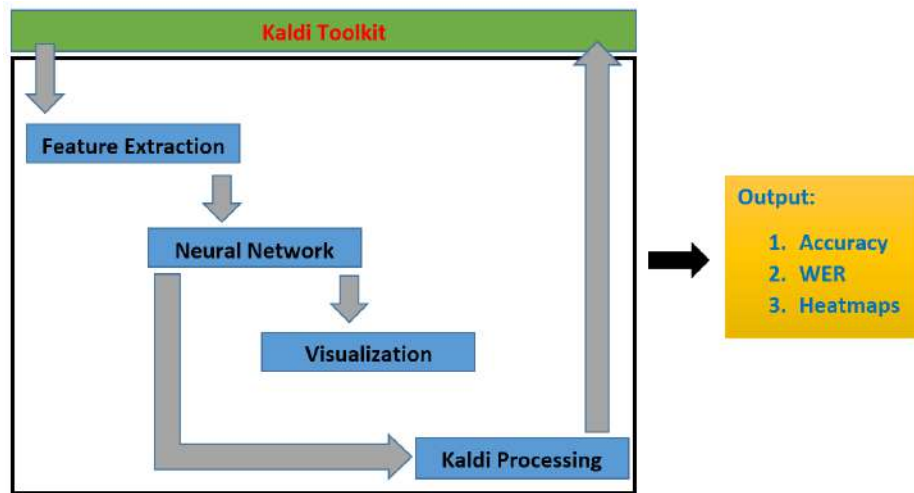


Figure 13: Component diagram of ‘deep-speech-vis’ framework

‘deep-speech-vis’ is a framework for **visualization** of automatic **speech** recognition task in **deep** neural networks. It is written on python programming

language. I have used Tensorflow library for neural network related implementations (Tensorflow is discussed in section 5.3). The components of the framework were created in such a way, so that they form a sequential pipeline, but can be run separately. So once a previous module of the pipeline (See figure 13 for the direction flow) is finished successfully, it is no longer needed to run it again for future processes, and the customization on the new module will not affect the previous results, as this new module can be run separately. This ensures parallel processing at its fullest speed, which is very crucial in deep learning paradigm because, the computation cost is considerably huge in terms of memory and GPU processing. Now I will briefly describe the individual components of this framework.

4.1 Input Module

Input module is where user can provide data location and custom directory locations according to their choice. It is totally separated from the inner modules, therefore the changes will not create any effect in the inner modules code. Neural network architecture related hyper-parameters and visualization related parameters can be customized from this module.

4.2 Feature Extraction Module

Feature extraction module converts the Kaldi text formatted ‘ark’ based feature data to ‘numpy’ based neural network suited data (Kaldi toolkit is discussed in section 5.1). It processes the data batch-wise, and stores it in a before-mentioned directory.

It also applies the cepstral mean and variance normalization (CMVN) to the input feature data. CMVN works as follows, after calculating the cepstrum values we subtract the average values of all coefficient values in an particular point from each coefficient, and divide the result by variance values of that point. CMVN helps to remove any channel effects (impulse

response of vocal tract, audio path, room, etc.), which provides better result in speech recognition task. [Viikki and Laurila (1998)]

This module also splices the data accordingly, based on the given context-window length. For example using ‘11 frame acoustic context-window’ means we concatenate feature vectors of 5 previous and 5 future frames to current frame’s vector, where the label is still center frame’s label. It has been shown that, using context-windows improves performance of DNN models [Hinton et al. (2012)]

It also generates the neural network output class data (batch-wise) from the Kaldi p.d.f. (probability density function) files.

4.3 Neural Network Module

Neural network module performs the neural network training and decoding procedures on the input features, and stores the result in terms of accuracy and different graphs in Tensorboard. It takes the HMM-GMM alignment (discussed in section 3.1) information as output, and feature information as input, and perform the training to improve the acoustic model and overall performance. The HMM-GMM alignment information is generated using Kaldi toolkit.

After the neural network decoding, it also creates the Kaldi based ‘ark’ files (for Kaldi based decoding of test inputs).

It is important to note that, although this module is based on Tensorflow library, but as it is a completely independent module, therefore any other library (ex. Theano, pytorch) based code can be used instead of this module’s code. Therefore the flexibility of using different libraries is well ensured here.

4.4 Visualization Module

Visualization module visualizes the utterance(s) according to the layerwise-relevance propagation (LRP) techniques (LRP is discussed in section 3.3)

in the forms of RGB (red, green and blue based) heatmaps. Currently it supports five different types of relevance types. They are as follows,

- Simple
- Flat
- WW
- Epsilon
- Alphabeta

All the layers of the neural network are visualized here for every epochs, so that one can observe the transitions in terms of the relevance score. For a particular relevance method in one epoch in each layer, it can create three types of images. They are as follows,

- Input image
- Relevance image
- Relevance on input image (to observe the relevance changes relatively, with a particular input layer)

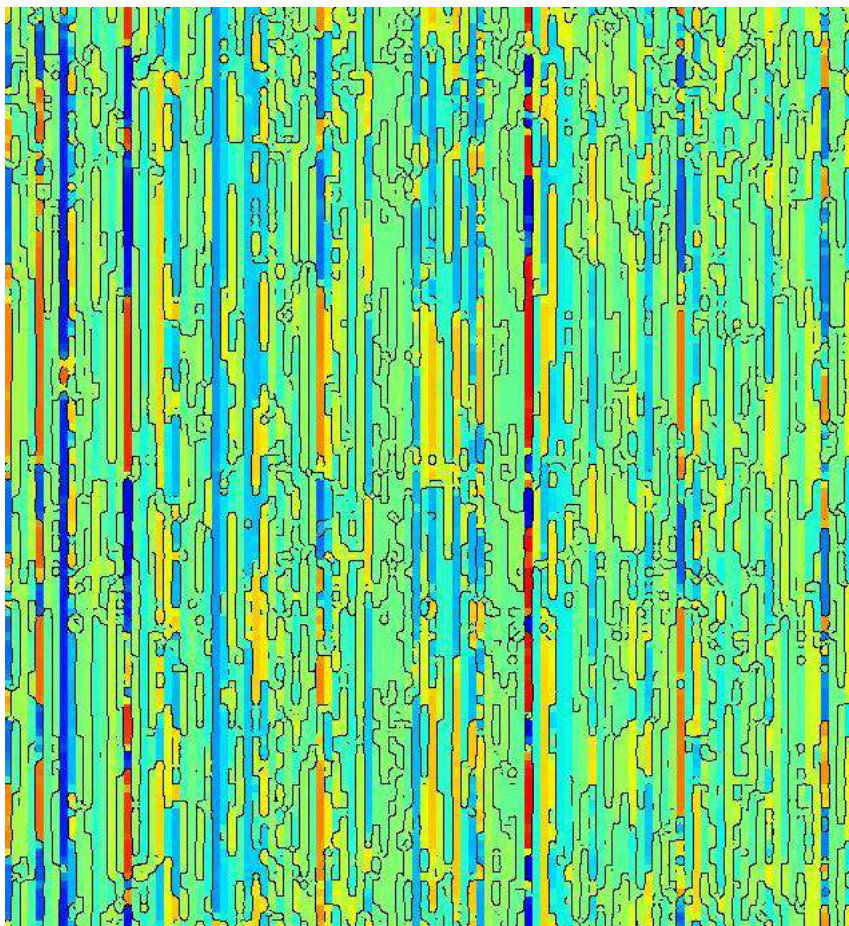


Figure 14: Relevance on input image for simple LRP method

Figure 14 shows an example of heatmap.

4.5 Kaldi Processing Module

Kaldi processing module decodes the ‘ark’ formatted data (generated in visualization module) to generate word error rate with the help of Kaldi toolkit. It has it’s own decoding shell script and ‘ark’ file generating program.

5 Resources

This section discusses about the toolkits, programming libraries, and dataset I have used to implement ‘deep-speech-vis’ framework, and to perform experiments on it. For creating HMM-GMM based automatic speech recognition system, I used Kaldi toolkit. To implement the feed-forward deep neural network, I used Tensorflow library. For experimenting, I was required to perform speech quality degradation to my input data. For this purpose, I used a large-scale open-source acoustic simulator. Finally, I used wall street journal (WSJ) dataset for all of my experiments.

5.1 Automatic Speech Recognition Toolkit: Kaldi

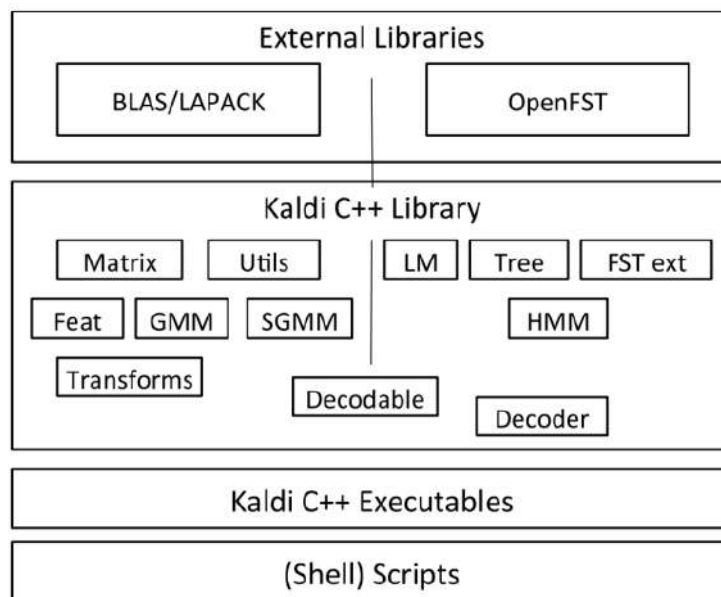


Figure 15: A diagram detailing the internal components of the Kaldi ASR system [Povey et al. (2011)]

Kaldi is a open source, has a very flexible license (Apache License v2.0) containing, and a highly customizable ASR toolkit Povey et al. (2011). The

framework uses the OpenFst finite-state transducer, which makes it usable on many different platforms. Most usefully, Kaldi contains several complete recipes for various types of ASR techniques. These allow to experiment with different approaches quite easily and effectively. Here I will briefly describe the components of the Kaldi toolkit, and how I used it in my system.

Components Of Kaldi

Figure 15 shows a schematic diagram of the Kaldi toolkit. Two sets of external libraries are used by Kaldi: OpenFst, which provides a finite-state framework, and the two linear algebra libraries: Basic Linear Algebra Subroutines (BLAS) and Linear Algebra PACKage (LAPACK).

The library modules are divided into two categories, where modules in a given category rely on only one of the two external libraries. Connecting these two categories is the module *DecodableInterface*.

Access to the different library functionalities is provided via a set of command-line tools. The creation and management of ASR systems is done using these tools. Each tool performs a specific functionality, such as accumulating statistics, summing accumulators, or updating a GMM-based acoustic model using maximum likelihood estimation. These functionalities can then be chained together via BASH (bourne again shell) scripting and input/output (I/O) redirection in order to form a complete ASR system.

In the preprocessing phase, Kaldi extracts mel frequency cepstral coefficients (MFCC) and perceptual linear prediction (PLP) features from the acoustic signals. Many customization options are provided, such as the number of melbins and the minimum and maximum frequency cutoffs. Kaldi supports several feature extraction approaches, including vocal tract length normalization (VTLN), cepstral mean and variance normalization (CMVN), linear discriminant analysis(LDA), semi-tied covariance/maximum likelihood linear transform (STC/MLLT), and heteroscedastic linear discriminant analysis (HLDA).

When constructing GMMs for the phones model, Kaldi supports both conventional GMM models (i.e. diagonal GMMs) and subspace gaussian mixture models (SGMMs). It can also be extended for new kinds of models. It is possible to separately specify the HMM topology for each context-independent phone. The topology format allows for non emitting states, and allows the user to pre-specify tying of the probability density functions (p.d.f.) s in different HMM states.

Kaldi represents its language models using its FST library. Thus, any language model representable as a finite state automaton – namely any regular language model – is representable by Kaldi. Included within Kaldi are a number of tools for deriving language model from corpora, and for importing language models from different formats.

Decoding is performed using decoding graphs, which are implemented using the FST library as weighted finite-state transducers. A number of separate decoders classes are provided, with some of the more sophisticated ones capable of on-the-fly language model re-scoring and lattice generation.

Data Pre-processing

In order to use WSJ corpus with Kaldi, some data pre-processing and formatting were required. For each experiment, I prepared three separate files to use by Kaldi:

- *text*
- *wav.scp*
- *utt2spk*

The text file consists of an utterance-by-utterance transcript of the corpus. This file contains every utterance matched with its text transcription. It consists of many lines, each with the format

```
utt_id WORD1 WORD2 WORD3 WORD4 ...,
```

where ‘utt_id’ denotes the utterance ID.

wav.scp contains paths to all audio files in the corpus. It consists of multiple lines of the format *file id/path/to/audio/file.wav*, where each path points to a single *.wav* audio file.

utt2spk contains a mapping from each utterance to its corresponding speaker. It is important to note that, in this context, the term speaker is often misrepresented, such that each recording session is often labeled as a different ‘speaker’. Therefore, the concept of ‘speaker’ does not have to strictly refer to a single person – it can be a room, accent, gender, or any common factor could influence the recording. When speaker normalization is performed, the normalization may actually be removing effects due to the recording quality or particular accent type. Choosing which criterion to use to determine this notion of ‘speaker’ is left to the modeler.

The file itself is a text file consisting of lines of the format

```
utt_id spkr
```

where ‘utt_id’ is the utterance ID and ‘spkr’ denotes speaker ID.

From these, I was able to generate all other necessary files using the scripts provided in Kaldi.

5.2 Speech Quality Degradation Toolkit: Acoustic Simulator

I have used an open-source acoustic simulator, created by [Ferras et al. \(2016\)](#), to create different degraded variants of clean input data. These variants are processed at 16kHz sample rate. They are as follows,

- Additive noise

- Telephone and audio codecs (compression/decompression module)
- Room and device impulse responses

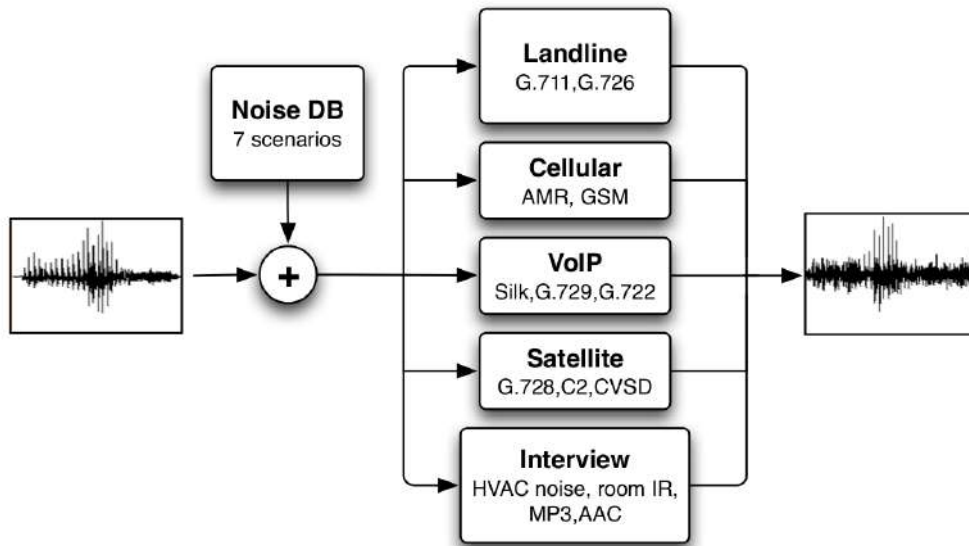


Figure 16: “Simplified block diagram of the environment simulator. A first module adds real noise recordings from a large noise database. The second module further degrades speech using telephone and interview conditions. [Ferras et al. (2016)]”

Figure 16 shows how a clean input data is being mixed with environmental noise, impulse response, and audio codec using this simulator.

60 hours of noise files were collected from the Freesound online archive (<https://freesound.org/>) in total around 1400 files. Each file is one to eight minutes long, and labeled manually with the noise condition. The noise files were added with the clean data at levels -15db lower. The categories of the noise files are as follows,

- Outdoors
- Public

- Private
- Transportation settings
- Babble
- Music
- Impulsive ambience noises (generated by the linear distortion simulation of different devices and rooms)

Different mainstream speech and audio codecs were also used to generate degraded audio. I am here briefly mentioning the major usage of these codecs to provide a general idea of the possible sound quality, when we use these to degrade the quality of the original sound data,

- **Landline telephony codecs**, telephone call sound
- **Cellular telephony codecs**, mobile phone call sound
- **Satellite/radio codecs**, frequency modulation (FM) radio or walkie-talkie sound
- **Voice over IP codecs**, Skype, Facebook Messenger call sound
- **MPEG Layer III (MP3) codec**, common sound from music players
- **Advanced Audio Codec (AAC) codec**, common audio recording sound in smart-phones

The simulator provides necessary scripts to perform the desired operations. Along with the possibilities of several combinations of different kinds of noise, the following settings are fixed considering their importance in speech recognition domain,

- Landline

- Cellular
- Satellite
- Voice over IP
- MPEG Layer III (MP3)
- Interview on clean and 15dB noisy conditions, generated by heating, ventilation, and air conditioning (HVAC) noise, small room reverberation and audio codecs

To ensure relative distribution of noisy feeling, bit-rate, dtx, packet loss, different noise recordings, and impulse responses are randomly sampled and mixed with the input data.

5.3 Deep Learning Library: Tensorflow

I have used Tensorflow to create my deep learning model. Different numerical calculations can be swiftly completed using Tensorflow. It is created and distributed by Google, and contains an Apache 2.0 open source license. Application program interface (API) is mainly provided for Python programming language. Tensorflow can be used and scaled effectively from small research and development tasks to large scale industrial tasks with distributive systems, and even in mobile devices. It can use both central processing unit (CPU) and Graphics Processing Unit (GPU) based systems effectively [Abadi et al. (2015)].

Computation in Tensorflow is expressed by the data flow and operations with the shape of a directed graph. The key structures are as follows,

- **Nodes:** Node does the computation and can possess zero or more inputs and outputs. Tensors are the data that moves between nodes, and can be viewed as multi-dimensional arrays of real values.

- **Edges:** The graph explains the propagation of data, branching, iterations, and updates to state. Special edges can be utilized to synchronize particular instance behavior inside of a graph.
- **Operation:** An operation is a named abstract computation, which can take input attributes, and produce output attributes. We can label an abstract computation, so that it can be recognized and re-used later.

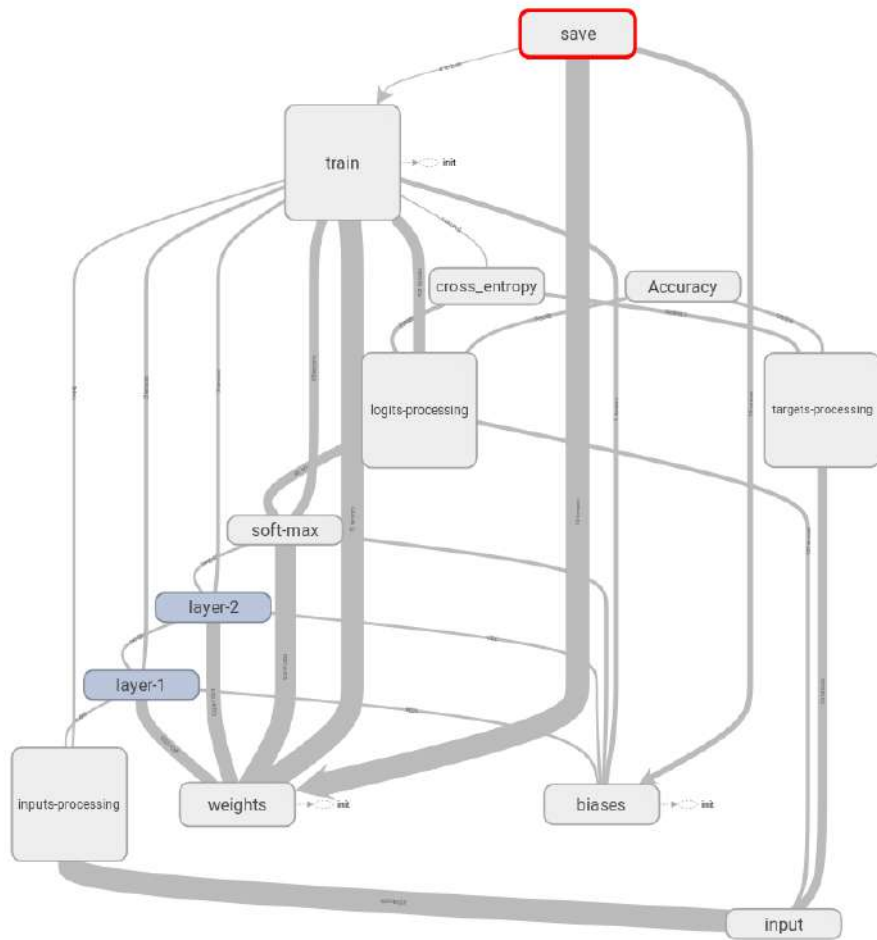


Figure 17: Graph of deep learning model in ‘deep-speech-vis’ framework

The positive aspects of using Tensorflow are, the ability of parallelizing tensor operations, automatic differentiation, and visualization tools (graph,

image, distribution, histogram, t-distributed stochastic neighbor embedding (t-SNE), principal component analysis (PCA) etc.) provided in Tensorboard. Figure 17 shows the graph representation of my deep learning model in Tensorboard.

5.4 Data Set: The Wall Street Journal Corpus

I have used wall street journal corpus (WSJ) [Paul and Baker (1992)] as the data for my experiments in ‘deep-speech-vis’ framework. It is taken from linguistic data consortium (LDC). LDC works to help on the linguistic data shortage problem for different research institutes, libraries, and universities. It provides various data sets.

Wall street journal is a famous international English newspaper from United States of America. WSJ corpus is built on the audio recitation in American English based pronunciation with machine readable text transcription from different articles of this newspaper. LDC provides two versions of it, CSR-I (WSJ0) Complete and CSR-II (WSJ1) Complete (CSR means continuous speech recognition). WSJ1 is in total length of 73 hours, and it contains 78000 individual utterances. Recording was done using two microphones (audio file extension ‘.wv1’ indicates first Sennheiser microphone, and ‘wv2’ indicates second microphone), and was sampled in 16000 sample rate using 1-channel pulse-code modulation (PCM) compression technique. The NIST (national institute of standards and technology) based file format contains many important information in its SPHERE headers. File-name conventions are as follows,

- **Speaker id**, first three bits from left
- **Speaking mode** (read-common, read-adaptation, read-common-vp), fourth bit
- **Speaker session number**, fifth to sixth bits

- **Session utterance number**, last two bits

The data is categorized in three different sets, training ('si284'), validation ('dev93'), and test ('eval92'). Training set is collected from 200 different speakers with 37393 utterances. Validation set is collected from 10 different speakers with 503 utterances. Test set is collected from 8 different speakers with 333 utterances. I have used training and test sets for my experiments.

6 Experiments And Results

In order to visualize the inner mechanism of neural networks during the training and testing phases, I conducted several automatic speech recognition task based experiments on different scenarios. At first I ran the GMM-HMM alignment using Kaldi, and decode it using WSJ test set to get the baseline performance value. Later I used a feed-forward neural network to improve the acoustic model performance, and observed the result by decoding it using Kaldi toolkit. Finally I visualized the mechanisms using a test utterance considering the following motivations; the reasoning of improved performance after each epoch, impact of noise in test data, impact of context-window while training the neural networks, and finally, the different recognition of vowels and consonants, by different nodes in the neural networks.

The traditional metrics to evaluate the performance of speech recognition task are accuracy and word error rate (WER). WER compares the total number of words in percentage with respect to the total number of substitutions, insertions, and deletions. Low WER indicates better performance for an ASR model. Accuracy is the ratio of total number of correct predictions over the total number of predictions. It is used for measuring the quality or improvement of acoustic model during the training of neural networks.

6.1 Kaldi Post-processing

When ‘deep-speech-vis’ framework starts its operations, It expects to get the input feature data as Kaldi ‘ark’ file in text format. It also expects the output labels as Kaldi p.d.f. based format. Traditional Kaldi toolkit provides generic scripts to compute and decode from HMM-GMM alignment. In these scripts the ‘ark’ files are generated in binary format, and the conversion of alignment information containing ‘ali’ files to p.d.f. files are done in run time, and Kaldi does not save the p.d.f. files. Therefore we need some post-processing works in Kaldi toolkit to ensure that, ‘deep-speech-vis’ gets the correct formatted input and output data.

I used the following instructions to convert the Kaldi binary formatted ‘ark’ based feature files to text based files,

```
# please run the commands from Kaldi s5 directory
. ./cmd.sh
. ./path.sh
. utils/parse_options.sh
train_cmd=run.pl
decode_cmd=run.pl

for i in {1..20} #number indicates total number of files
do
copy-feats ark:data/test_eval92/data_fbank_40
/raw_fbank_test_eval92.$i.ark ark
,t:- >> 'your-directory'/test-feats-ark.txt
done
```

I used the following instructions to convert the Kaldi ‘ali’ files to Kaldi p.d.f. files,

```
# please run the commands from Kaldi s5 directory
```

```

. ./cmd.sh
. ./path.sh
. utils/parse_options.sh
train_cmd=run.pl
decode_cmd=run.pl

for i in {1..10} #number indicates total number of files
do
gunzip -c exp/tri2b_ali_si284/ali.$i.gz
| ali-to-pdf exp/tri2b_ali_si284/final.mdl ark:- ark,t:-
| gzip > exp/tri2b_ali_si284/pdf.$i.gz

```

6.2 Neural Network Implementation

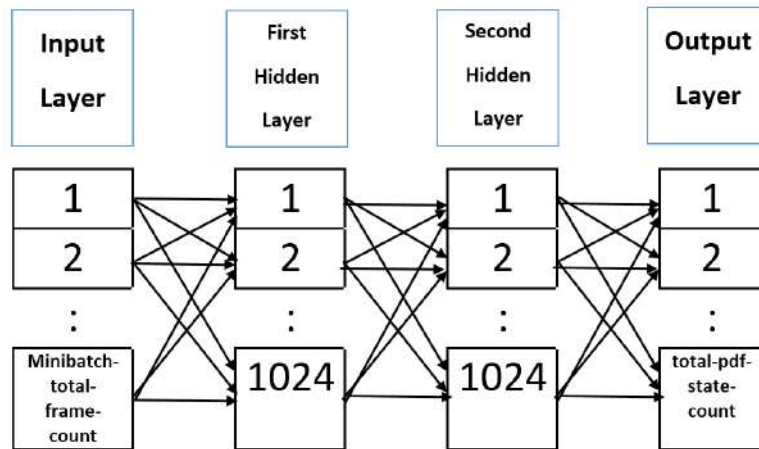


Figure 18: Neural network architecture

I used a feed forward neural network with two hidden layers for the experiments. Figure 18 shows the graph architecture of this network. It is worth mentioning that, unlike the conventional ‘zero-padding’ in each mini-batches; I have concatenated all the utterances together for a particular mini-batch

in this network. I did this because, it has been shown that, ‘zero-padding’ works better in CNN but in MLP the concatenation provides better results.

Table 1 provides the parameters I have used for this neural network.

Name	Value
Initial learning rate	0.001
Number of neurons in first hidden layer	1024
Number of neurons in second hidden layer	1024
Running Epochs	10
Number of validation mini-batches	2
Context-window frame length	11
Number of utterances per mini-batch	64
Activation function	tanh
Output layer classification function	softmax
Optimizer algorithm for learning	Adam optimizer

Table 1: Neural Network Parameters

6.3 Experimental Setup

Now I will discuss about the experiments in the following sections. After the discussion I will provide the results based on WER and accuracy for the experiments. I will also provide different heatmaps generated from the final hidden layers for different LRP methods. To compare equivalently, all the calculation of relevance scores, and generation of different LRP methods based heatmaps, are computed from the same utterance. Table 2 provides information about speech data used in the experiments.

Name	Value
Maximum length of input frames of a training utterance	2433
Total number of training utterances	37393
Maximum length of output labels of a training utterance	2433
Total number of training labels	3383
Total number of training mini-batches	584
Input dimension (with context-window)	440
Input dimension (without context-window)	40
Maximum length of input frames of a test utterance	1447
Total number of test utterances	333

Table 2: Details about speech data used in experiments

6.3.1 Experiment 1: Epoch Change Visualization

In this experiment I used original WSJ data (I am referring this as ‘clean’ data) for training the model. Then I calculated relevance scores after decoding a particular utterance of ‘clean’ data after each epoch, and tried to show the difference in the generated heatmaps, based on these scores. As I have considered almost all the LRP methods, so I am only providing the heatmaps after epoch 1 and 10 here. Otherwise the total number of heatmaps will be considerably large.

6.3.2 Experiment 2: Context-Window Visualization

In this experiment I used the ‘clean’ data to train the model. I ran the experiment without performing context-window operation (discussed in section 4.2), and generated heatmaps based on that. Then I compared these heatmaps with experiment 1, to observe the importance of using context-window in deep neural networks. I am only providing the heatmaps after epoch 10 here.

6.3.3 Experiment 3: Noisy Data Decoding Visualization

In this experiment I degraded speech quality of the ‘clean’ data from test data set using acoustic simulator (the simulator is discussed in section 5.2). I am referring this degraded data as ‘noisy’ data. Then I perform the decoding operation using one utterance of this ‘noisy’ data with the training model created by the ‘clean’ data in experiment 1. Then I generated heatmaps to observe the impact of noise in automatic speech recognition task. I am only providing the heatmaps after epoch 10 here.

6.3.4 Experiment 4: Vowels And Consonants Visualization

In this experiment I tried to visualize different vowels and consonants, and tried to observe their differences. For each phone I took features of 1135 input utterance frames of the same phone label from ‘clean’ data for this experiment. I considered four most frequent vowels and consonants from WSJ data set. I am only providing the heatmaps after epoch 10 here.

Phone	Example	Frequency
EH1_I	Ed	525240
AE1_I	at	400965
IH0_I	it	383469
AA1_I	odd	362586
Z_E	zee	743694
N_I	knee	670705
T_I	tea	582537
S_B	sea	576350

Table 3: Vowel and Consonant Phones used in experiment 4

Table 3 provides the details about these phones. The naming convention of the phones is based on ARPAbet symbol set where vowels can carry the following lexical stress markers,

- **0**: No stress
- **1**: Primary stress
- **2**: Secondary stress

Position of the phone in the word is denoted using the following notations,

- **_B**: Beginning
- **_E**: End
- **_I**: Word-internal phone
- **_S**: Singleton (word with one sound only)

6.4 Heatmaps And Discussion

Heatmaps represent the relevancy score of the neurons for all the input frames in aspect with the classification result. I have scaled the score 100 times to represent the color difference more brightly. Here each row of the heatmap represents a frame of input features, and each column represents a particular neuron. To visualize it clearly, I enlarge each data point (1 pixel) to a 6 * 6 pixel matrix.

Heatmaps generated by WW LRP and alphabeta LRP (with alpha value 0.7) work considerably better than others. Epsilon LRP performs poorly. I think I need to experiment with several values for epsilon parameters in future to get a suitable heatmap condition for speech data, as epsilon LRP was originally intended for computer vision related data.

After providing the heatmaps I have discussed about them in the subsequent sections.

Epoch	Average Cost	Training Accuracy	Validation Accuracy
001	2.923	0.419	0.417
002	2.217	0.457	0.450
003	2.051	0.479	0.467
004	1.951	0.493	0.477
005	1.881	0.504	0.485
006	1.828	0.511	0.491
007	1.786	0.518	0.496
008	1.752	0.523	0.500
009	1.723	0.528	0.504
010	1.698	0.532	0.506

Table 4: Epoch Wise accuracy in clean data with context-window

6.4.1 Experiment 1

Table 4 is showing the epoch wise accuracy data. The **WER** for this experiment is **5.56%** [314 / 5643, 26 insertions, 40 deletions, 248 substitutions] which is slightly worse then Kaldi decoding **WER** of **5.16%** [291 / 5643, 52 insertions, 22 deletions, 217 substitutions].

Heatmaps: Alphabet LRP

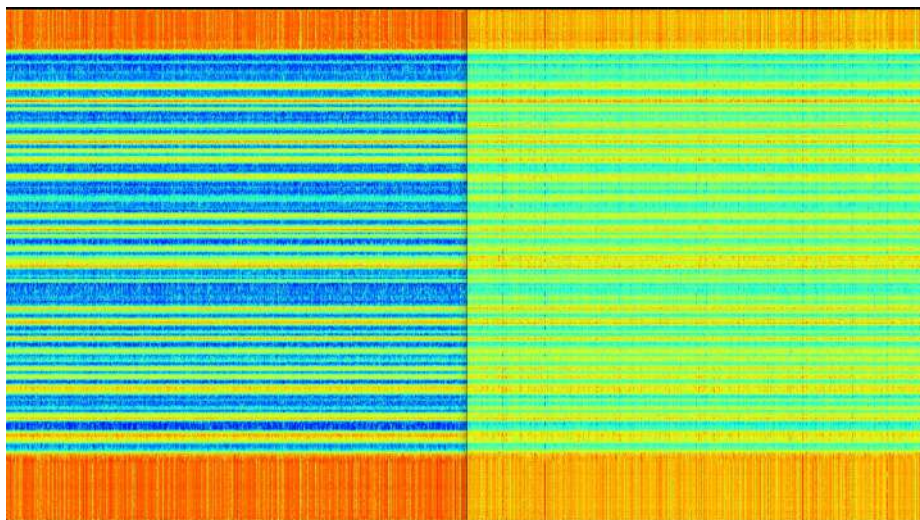


Figure 19: Experiment 1: Alphabet LRP, Left- After Epoch: 1, Right- After Epoch: 10

Flat LRP

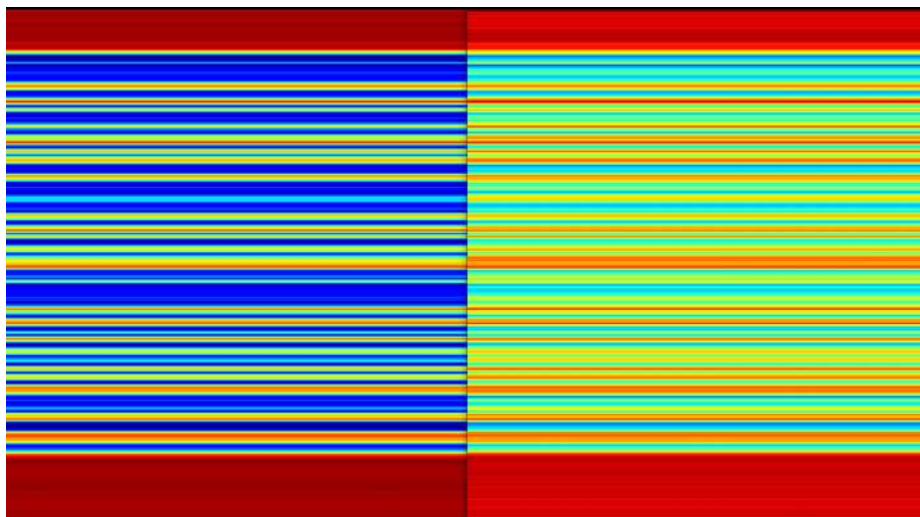


Figure 20: Experiment 1: Flat LRP, Left- After Epoch: 1, Right- After Epoch: 10

WW LRP

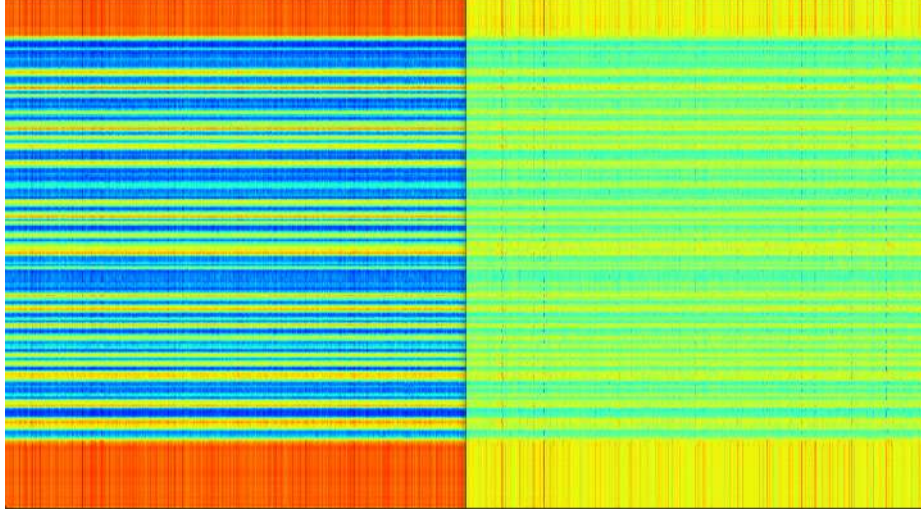


Figure 21: Experiment 1: WW LRP, Left- After Epoch: 1, Right- After Epoch: 10

Discussion On Experiment 1

Experiment 1 shows that, initially all the neurons regard all the input frames with almost similar level of relevancy. This is why initial heatmaps are very bright and messy in pattern. After each epoch the neurons refine themselves, and finally regard only certain part as relevant. This means initially all the neurons work almost equally for all classes, but later, a certain group of neurons work for detecting a particular class from the input. So later, we can see images are less bright with more clear patterns. This observation will be much clear in experiment 4.

6.4.2 Experiment 2

Table 5 is showing the accuracy transition for experiment 2. The **WER** for this experiment is **15.61%** [881 / 5643, 58 insertions, 212 deletions, 611 substitutions].

Epoch	Average Cost	Training Accuracy	Validation Accuracy
001	4.627	0.177	0.176
002	4.319	0.178	0.178
003	4.232	0.184	0.184
004	4.171	0.188	0.189
005	4.125	0.191	0.191
006	4.088	0.194	0.194
007	4.059	0.196	0.197
008	4.035	0.198	0.198
009	4.014	0.199	0.199
010	3.996	0.201	0.200

Table 5: Epoch Wise accuracy in clean data without context-window

Heatmaps: Alphabeta LRP

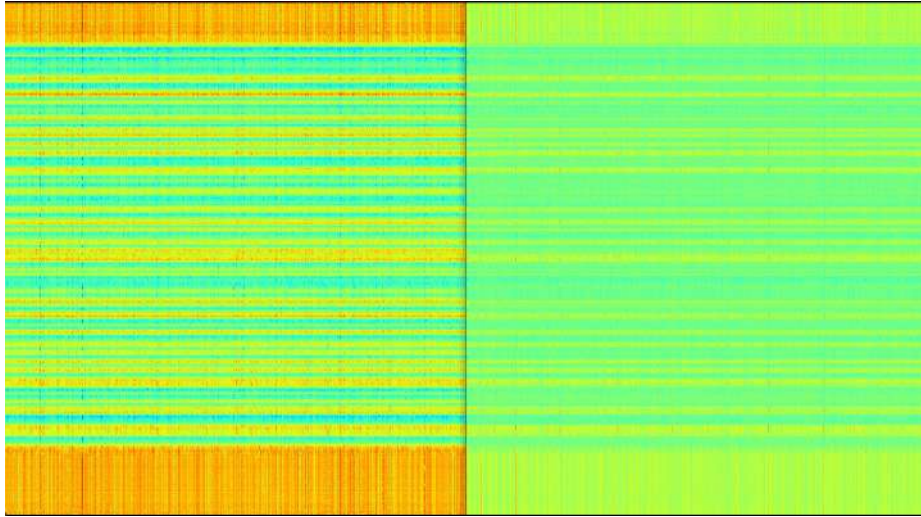


Figure 22: Experiment 2: Alphabeta LRP, After Epoch: 10, Left-With Context-Window, Right-Without Context-Window

Simple LRP

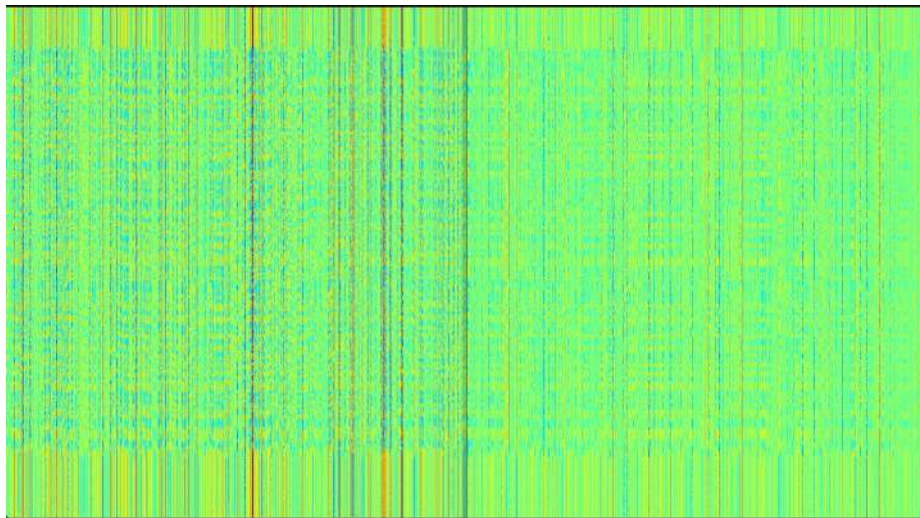


Figure 23: Experiment 2: Simple LRP, After Epoch: 10, Left-With Context-Window, Right-Without Context-Window

WW LRP

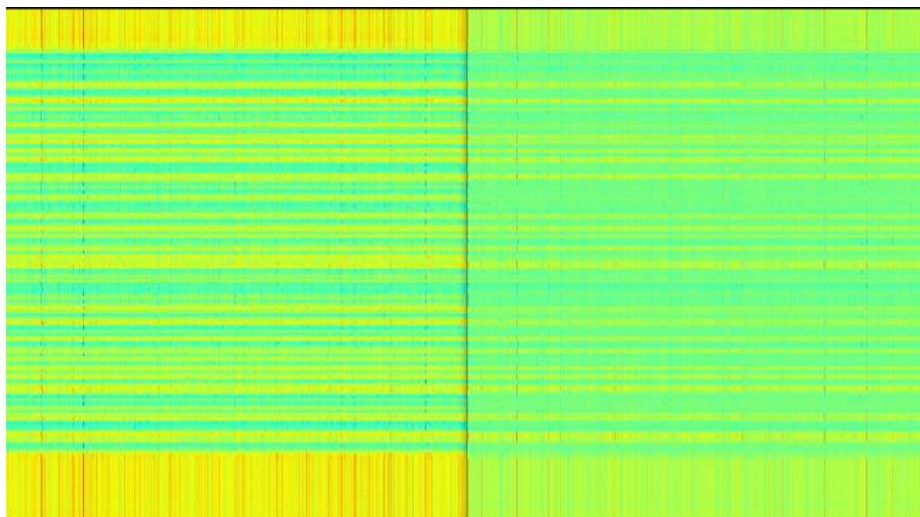


Figure 24: Experiment 2: WW LRP, After Epoch: 10, Left-With Context-Window, Right-Without Context-Window

Discussion On Experiment 2

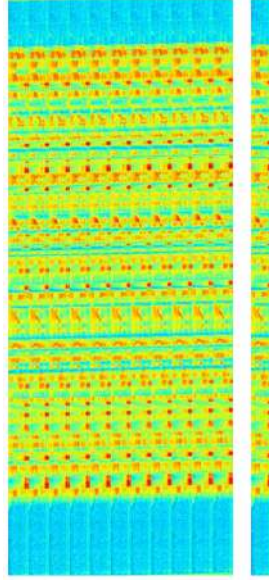


Figure 25: Heatmaps of one test input utterance; left one is with context-window, right one is without context-window

Figure 25 shows the test input utterance in the form of RGB valued heatmaps. Right one is without context-window and left one is with context-window. Here we can see that the data patterns are same, but right heatmap contains more of the same patterns. So we can assume that, ‘with context-window’ provides more area to the neurons, so they can map it more efficiently. Experiment 2 supports this assumption. We can see that, ‘with context-window’ provides more refined and grouped relevance visibility of the neurons, comparing with the ‘without context-window’ case.

6.4.3 Experiment 3

The WER details for this experiment are as follows:

96.14% [5425 / 5643, 48 insertions, 3937 deletions, 1440 substitutions]

Heatmaps: Alphabeta LRP

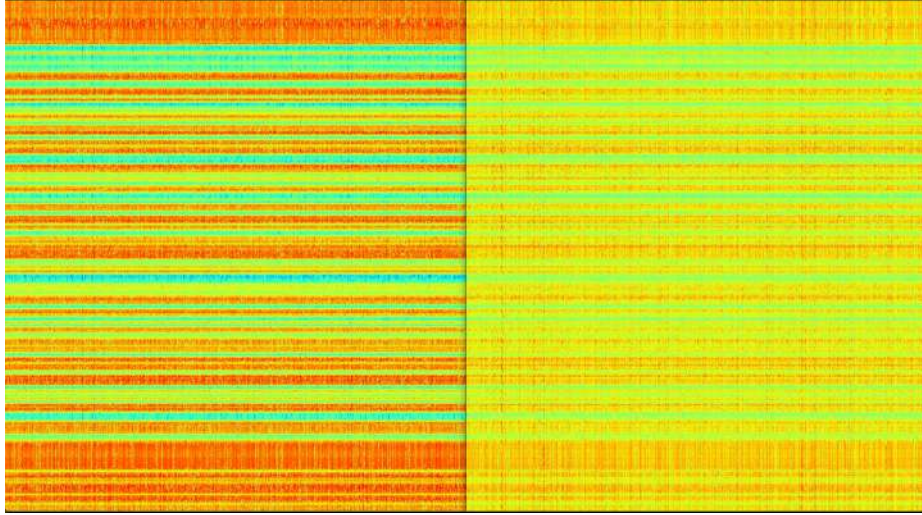


Figure 26: Experiment 3: Alphabeta LRP, After Epoch: 10, Left-Clean Data, Right-Noisy Data

WW LRP

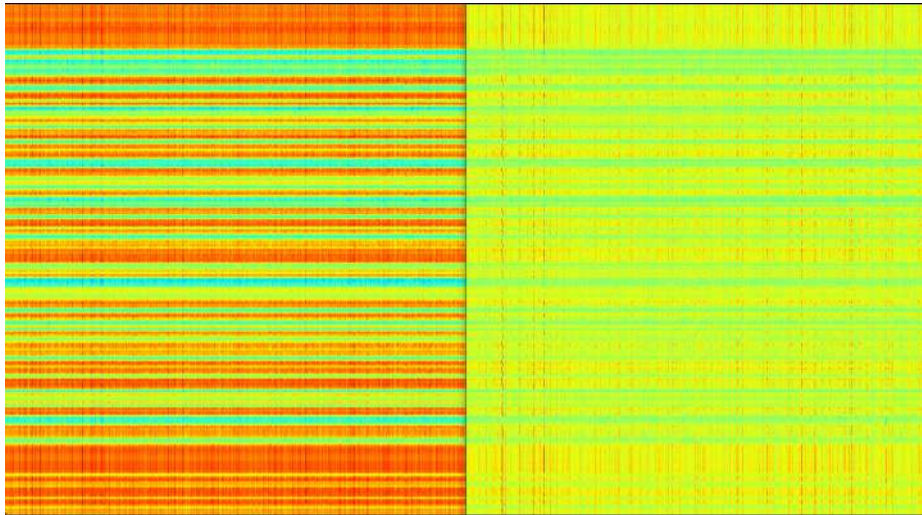


Figure 27: Experiment 3: WW LRP, After Epoch: 10, Left-Clean Data, Right-Noisy Data

Discussion On Experiment 3

Experiment 3 provides us some insight on why it is very hard to decode ‘noisy’ data accurately. The patterns are considerably different in ‘clean’ data which shows that it is much easier for the neurons to form groups in clean data. We can see lots of color variation on ‘clean’ data, which supports this notion. Therefore, performance is better on ‘clean’ data. We need to investigate more in future. The very poor WER suggests some rechecking with the whole training hyper-parameters procedure.

6.4.4 Experiment 4

Heatmaps: Vowel Phone: AA1_I

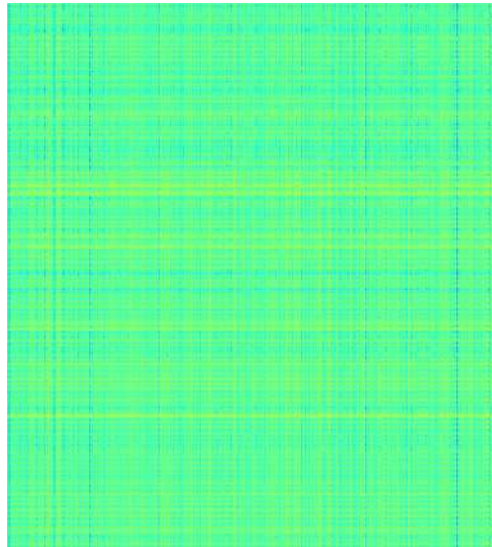


Figure 28: Experiment 4: WW LRP, Vowel phone AA1_I, After Epoch: 10

Vowel Phone: IH0_I

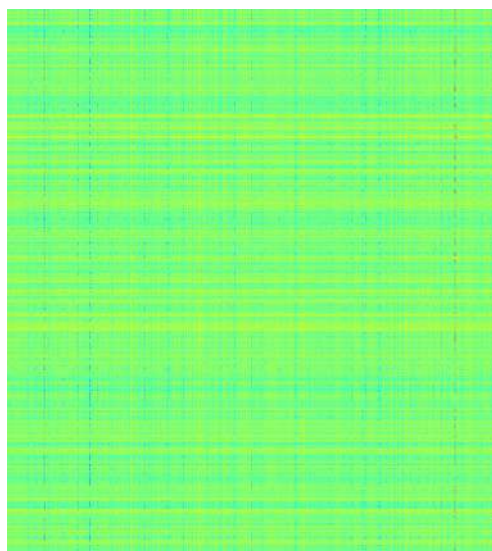


Figure 29: Experiment 4: WW LRP, Vowel phone IH0_I, After Epoch: 10

Vowel Phone: AE1_I

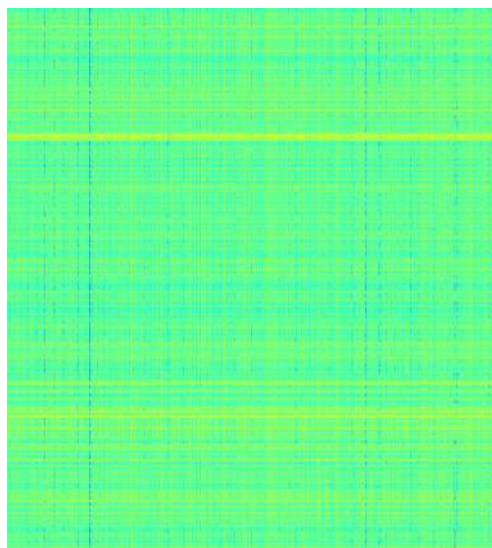


Figure 30: Experiment 4: WW LRP, Vowel phone AE1_I, After Epoch: 10

Vowel Phone: EH1_I

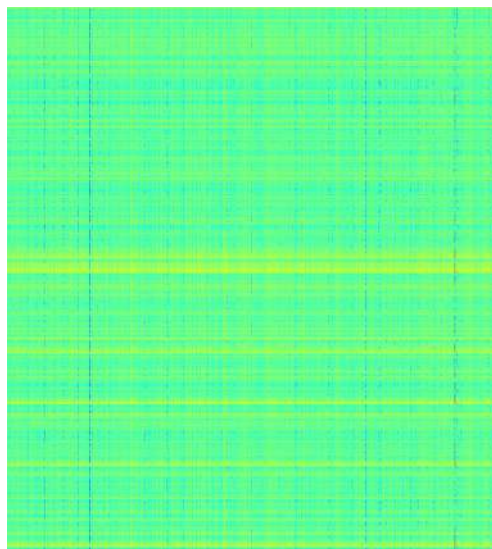


Figure 31: Experiment 4: WW LRP, Vowel phone EH1_I, After Epoch: 10

Consonant Phone: Z_E

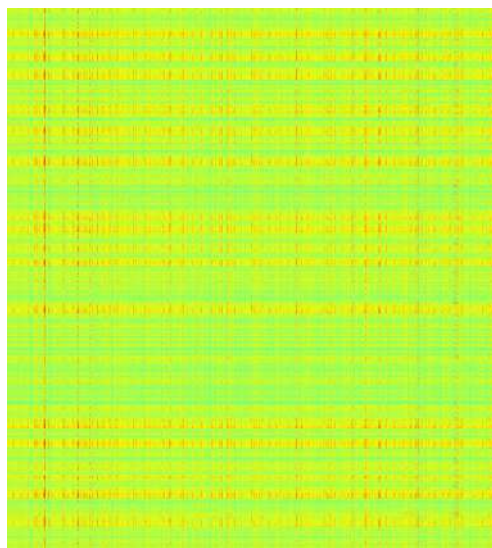


Figure 32: Experiment 4: WW LRP, Consonant phone Z_E, After Epoch: 10

Consonant Phone: N_I

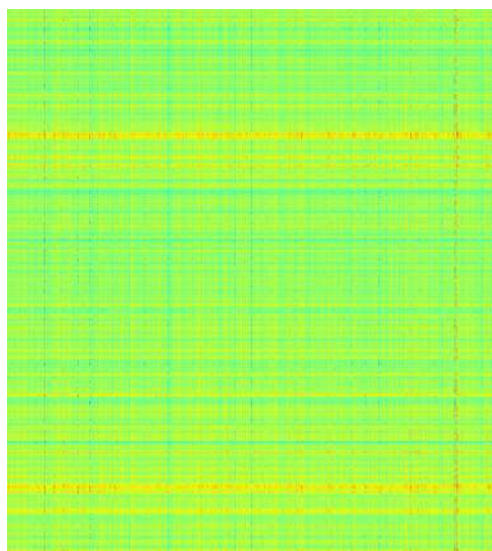


Figure 33: Experiment 4: WW LRP, Consonant phone N_I, After Epoch: 10

Consonant Phone: T_I

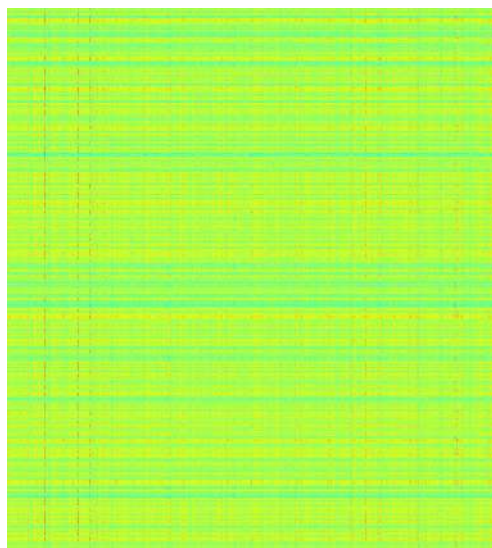


Figure 34: Experiment 4: WW LRP, Consonant phone T_I, After Epoch: 10

Consonant Phone: S_B

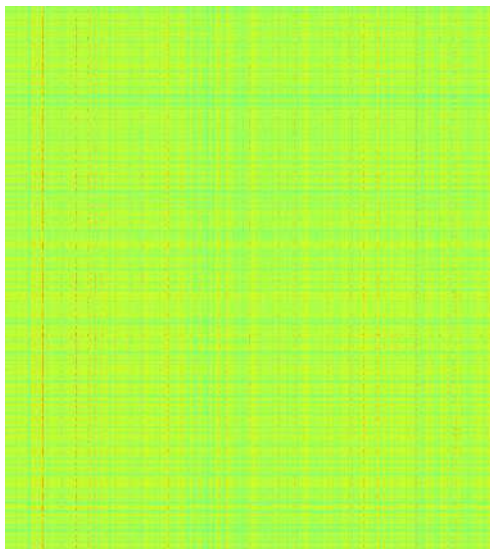


Figure 35: Experiment 4: WW LRP, Consonant phone S_B After Epoch: 10

Discussion On Experiment 4

Experiment 4 provides us a clear difference on visualization of vowels and consonants. The consonant heatmaps are considerably brighter than the vowel heatmaps. It clearly shows that vowels and consonants are recognized by different groups of neurons in neural network. In experiment 1 we are considering all types of phones in a utterance, but in experiment 4 we are considering only one phone for each heatmap. So in experiment 4, the heatmaps are more homogeneous in pattern. It means neural network can distinguish vowels and consonants without having any explicit information. This is very interesting and should be explored further for different linguistic aspects (formant values, articulation places etc.).

Considering the various colors in the heatmaps of experiment 1, we can see that, in experiment 4 the color variation is much less, specially for the vowels. It supports the phonetic notion because, vowel sound signals are less noisy in nature. However, it also raises another question. The color in the vowels

should not be changed as we are seeing for some vowels. My assumption is, it is due to the alignment issue from Kaldi. Kaldi alignments are not always accurate. There is another point to consider, I have considered random 1135 frames for each phone. All of them may not be the best examples for a particular phone. Human voice generation of vowels and consonants is also not same for all the utterances.

I have also performed some analysis on the relevance scores for phoneme recognition. I have used WW LRP method to generate relevance score. I have scaled the mean value by multiplying it with -100, as the initial relevance score values were negative (log values of floating point numbers). Therefore higher the value in the graph, lower the relevancy score value.

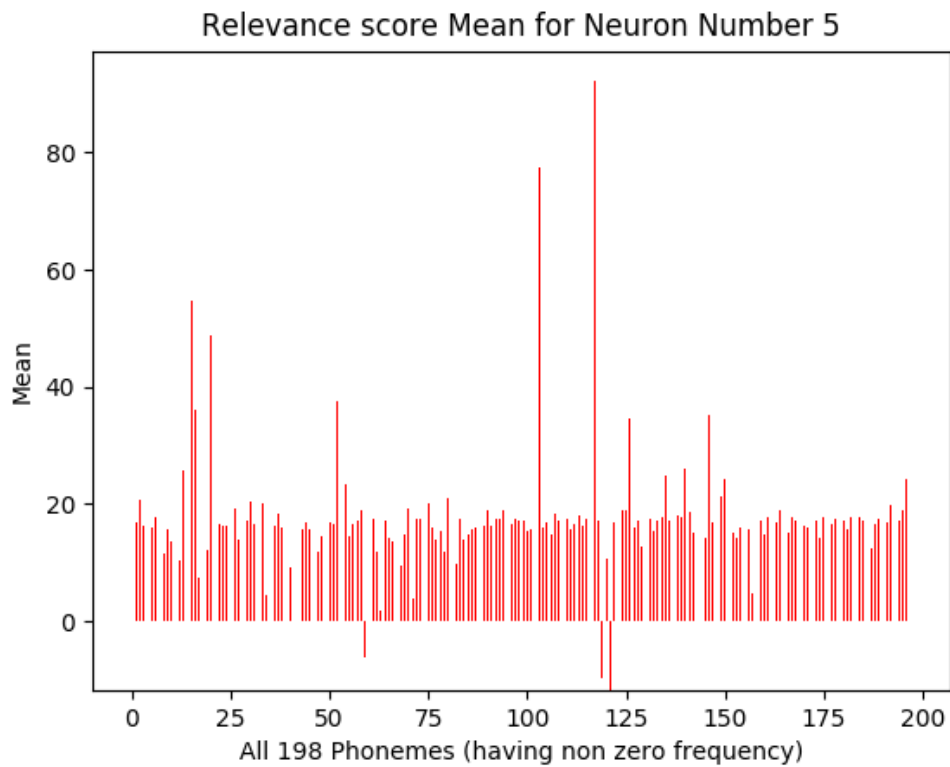


Figure 36: Experiment 4: Analysis of a random neuron (5) of final layer

Figure 36 shows the mean relevance scores of all the phones having a nonzero frequency on a random neuron. We can clearly see that relevance scores varies for different phones.

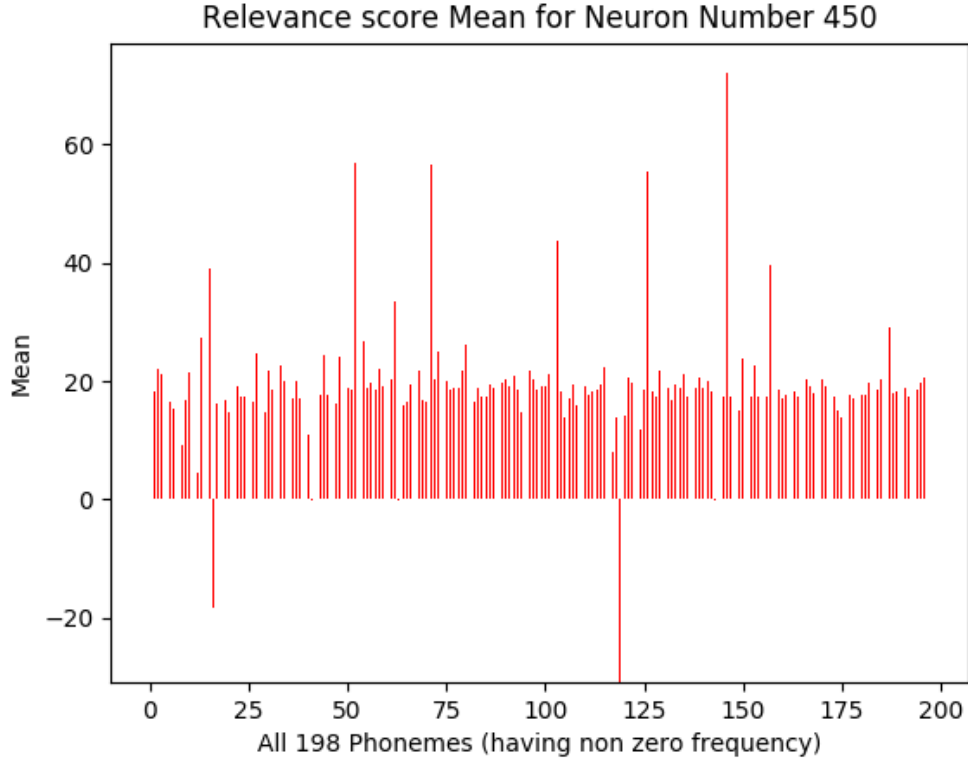


Figure 37: Experiment 4: Analysis of a random neuron (450) of final layer

Figure 37 is showing different distributions of mean relevance scores comparing with figure 36. Therefore it is evident that relevance scores are not same for all the neurons. These two graphs also show that, some neurons are extremely relevant to identify some particular phones. Therefore it also supports our previous observation from heatmaps that, some neurons are mainly responsible for identifying a particular phone.

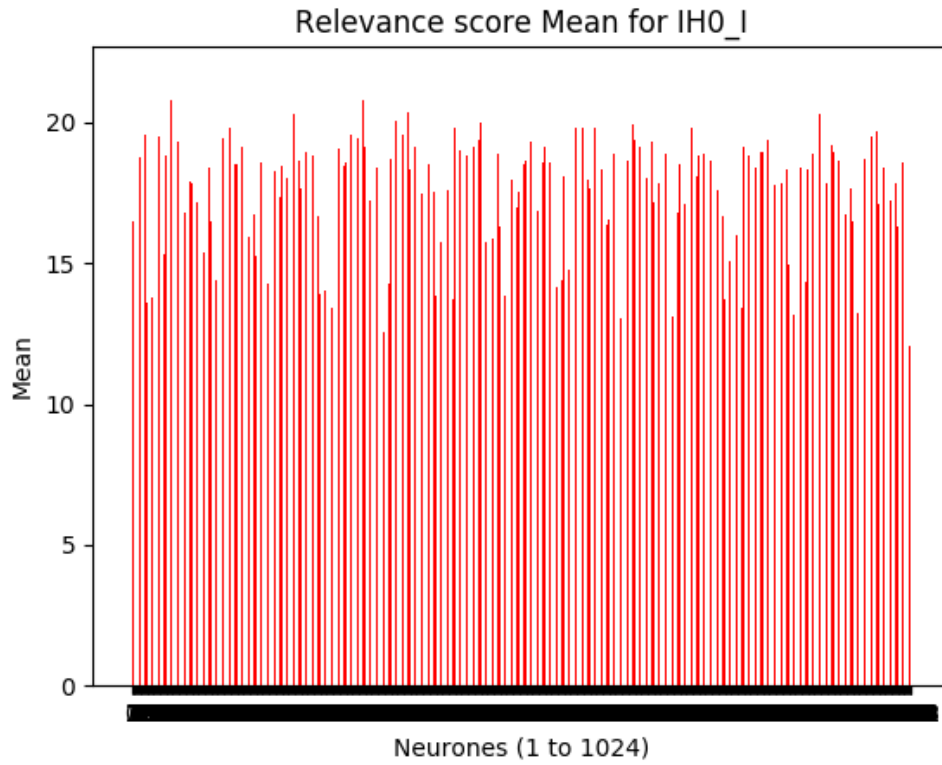


Figure 38: Experiment 4: Relevance score analysis of a vowel with all the neurons of final layer

Figure 38 is showing the relevance score for a particular vowel. Again it is noticeable that these relevance scores are varying across the neurons.

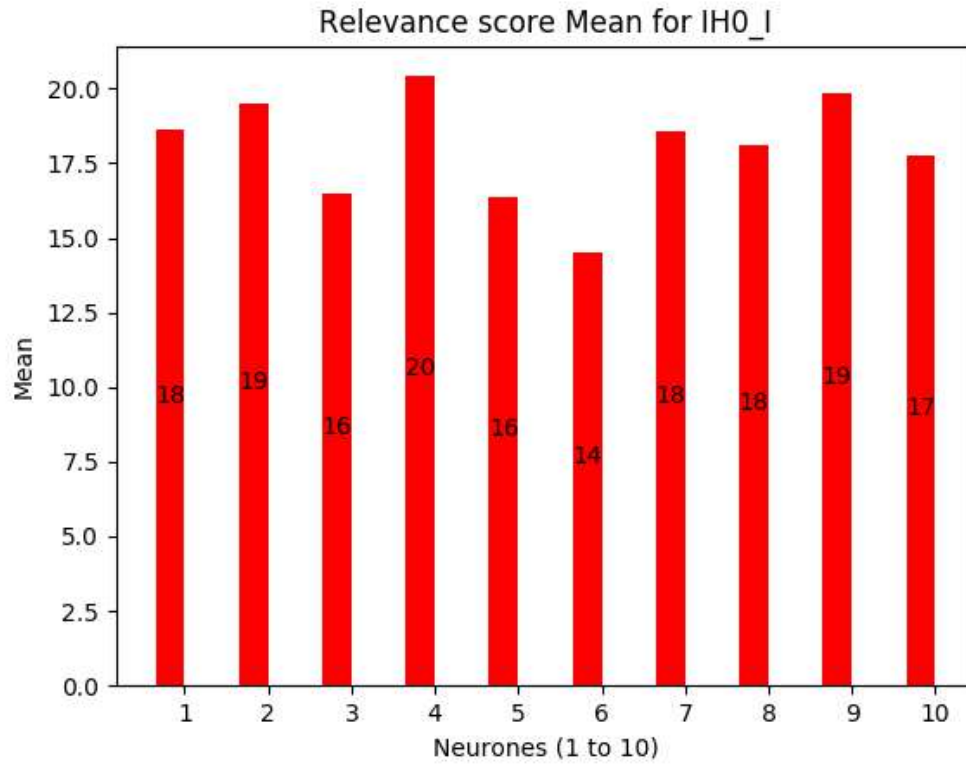


Figure 39: Experiment 4: Relevance score analysis of a vowel with first 10 neurons of final layer

To make it a bit simple in figure 39 we are only considering first 10 neurons of the final layer. We can see that the mean relevance scores vary.



Figure 40: Experiment 4: Relevance score analysis of a consonant with all the neurons of final layer

Figure 40 and figure 41 are showing the relevance scores for a particular consonant across the neurons. We can notice some differences here comparing with the vowel's distribution.

We have got some interesting insights with evidence about hidden layer mechanisms in these experiments. Comparing with computer vision domain, the interpretability is not that much obvious in automatic speech recognition task. We have to fine tune the parameters in LRP methods. The phone labeling is also not as accurate as the pixels of the image classification tasks. The huge dimensionality is also another major issue. We need to focus on these aspects to make visualization more effective in automatic speech recognition

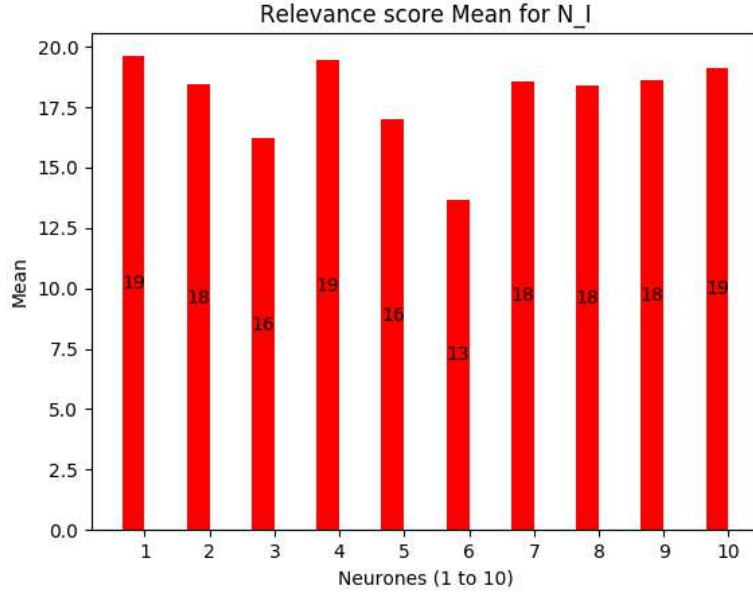


Figure 41: Experiment 4: Relevance score analysis of a consonant with first 10 neurons of final layer

domain.

7 Conclusion

In this work I have reviewed the recent visualization techniques of computer vision domain for kernels and deep neural networks extensively, and applied the most suitable technique, LRP method in ASR domain for the first time. This work also attempts to make the deep learning hidden layer systems more interpretable, by adding another visualization tool in the whole process chain. I have observed the hidden layer mechanisms such as learning after each epoch change, effectiveness of context-window, impact of speech quality degraded data in decoding, and the difference between identifying vowels and consonant phones. Each of these different LRP methods provides unique opportunities to view the classification problems in deep learning paradigm

more robustly; specially in natural language processing paradigm, as I have shown it in automatic speech recognition domain. Therefore it can increase the possibilities of many more observations, outcomes for us in future.

8 Future Work

In this work I have generated the heatmaps which are to understand intuitively. As LRP methods are completely quantifiable, therefore in future I will try to add the measuring score feature module to the ‘deep-speech-vis’ framework. Different neural network models (ex. CNN, RNN-LSTM, generative adversarial networks (GANs) etc.) and activation functions should be tested with these heatmaps, to identify the differences, and to get the insights more deeply. If we can handle the computational complexity of huge dimensionality management problem in terms of processing power, then we can focus in the micro areas also. Phoneme recognition and classification based on LRP heatmaps can be one of these micro areas. Difference among vowels based on formant values, difference among consonants based on articulation points, and whether these differences matches with relevance score, should be examined also. Therefore it can provide us opportunities to venture and explore linguistics based ideas in the phonetics section. Using spontaneous speech, we can investigate individual language specific phonetic classification more robustly, by utilizing these LRP methods, which will ensure less bias, and more accurate classification. Finally we can generate synthesized voice, with a lot less computational effort, by using this technique.

Appendix I: Acronyms

AI Artificial Intelligence

AM Acoustic Model

API Application Program Interface

ARPAbet set of phonetic transcription codes developed by Advanced Research Projects Agency (ARPA)

ASR Automatic Speech Recognition

BASH Bourne Again Shell

BLAS Basic Linear Algebra Subroutines

BoW Bag of Words

CMVN Cepstral Mean and Variance Normalization

CNN Convolutional Neural Network

codec Compression/Decompression Module

CPU Central Processing Unit

CSR Continuous Speech Recognition

deep-speech-vis framework for VISualization of automatic SPEECH recognition task in DEEP neural network

DNN Deep Neural Network

FST Finite State Transducer

GAN Generative Adversarial Network

GMM Gaussian Mixture Model

GPU Graphics Processing Unit

HLDA Heteroscedastic Linear Discriminant Analysis

HMM Hidden Markov Model

HVAC Heating, Ventilation, and Air Conditioning

I/O Input/Output

LAPACK Linear Algebra PACKage

LDA Linear Discriminant Analysis

LDC Linguistic Data Consortium

LM Language Model

LRP Layer-wise Relevance Propagation

LSTM Long Short-Term Memory recurrent neural network

MFCC Mel Frequency Cepstral Coefficients

MLLT Maximum Likelihood Linear Transform

MLP Multilayer Perceptron

MP3 Moving Picture Experts Group Layer-3 Audio

NIST National Institute of Standards and Technology

PCA Principal Component Analysis

PCM Pulse-Code Modulation

PDF Probability Density Function

PLP Perceptual Linear Prediction

ReLU Rectified Linear Unit

RGB Red Green Blue

STC Semi-Tied Covariance

SVM Support Vector Machine

t-SNE t-Distributed Stochastic Neighbor Embedding

VoIP Voice over Internet Protocol

VTLN Vocal Tract Length Normalization

WER Word Error Rate

WSJ Wall street journal

Appendix II: Mathematical Reference

In this section I am providing a concise representation and explanation of different important mathematical concepts I have used. The following sub-sections are taken from [Raschka \(2017\)](#).

Sets and Intervals

$\mathbb{R}$ set of real numbers

Sequences

$\sum_{i=0}^n x_i$ summation of an indexed variable x_i defined as
$$\sum_{i=0}^n x_i = x_0 + x_1 + \dots + x_n$$

Functions

$f : A \rightarrow B$ function f with domain A and codomain B
 $\arg \max_x f(x)$ the x value that makes $f(x)$ as large as possible

Linear Algebra

x scalar (lower-case italics notation)
 $\mathbf{x}$ column vector (lower-case bold notation)
 $\mathbf{X}$ matrix (upper-case bold notation)
 $\mathbb{R}^n$ real coordinate space, written as a column vector with length n
 $\mathbf{A} \odot \mathbf{B}$ hadamard product, element-wise matrix multiplication

Calculus

$\frac{\partial f}{\partial x}$ partial derivative of $f(x, y, \dots)$
with respect to variable x , where x is a scalar

Probability

$P(A|B)$ conditional probability of A given B

Approximation

$\approx$ approximate equality, for instance, $e \approx 2.71828$ is the approximation of Euler's number.

Logic

$\forall$ universal quantifier, means for all.

For example, " $\forall x \in \mathbb{R}, x > 1$ "

translates to "for all real numbers x , x is greater than one"

References

- Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dan Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL <https://www.tensorflow.org/>. Software available from tensorflow.org.
- Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one*, 10(7):e0130140, 2015.
- David Baehrens, Timon Schroeter, Stefan Harmeling, Motoaki Kawanabe, Katja Hansen, and Klaus-Robert MÅzller. How to explain individual classification decisions. *Journal of Machine Learning Research*, 11(Jun):1803–1831, 2010.
- Mikio L Braun, Joachim M Buhmann, and Klaus-Robert MÅzller. On relevant dimensions in kernel feature spaces. *Journal of Machine Learning Research*, 9(Aug):1875–1908, 2008.
- Alexey Dosovitskiy and Thomas Brox. Inverting convolutional networks with convolutional networks. *CoRR abs/1506.02753*, 2015.
- Xuedan Du, Yinghao Cai, Shuo Wang, and Leijie Zhang. Overview of deep learning. In *Chinese Association of Automation (YAC), Youth Academic Annual Conference of*, pages 159–164. IEEE, 2016.

- Marc Ferras, Srikanth Madikeri, Petr Motlicek, Subhadeep Dey, and Hervé Bouchard. A large-scale open-source acoustic simulator for speaker recognition. *IEEE Signal Processing Letters*, 23(4):527–531, 2016.
- Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep learning*. MIT press, 2016.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- Katja Hansen, David Baehrens, Timon Schroeter, Matthias Rupp, and Klaus-Robert Müller. Visual interpretation of kernel-based prediction models. *Molecular Informatics*, 30(9):817–826, 2011.
- Geoffrey Hinton, Li Deng, Dong Yu, George E Dahl, Abdel-rahman Mohamed, Navdeep Jaitly, Andrew Senior, Vincent Vanhoucke, Patrick Nguyen, Tara N Sainath, et al. Deep neural networks for acoustic modeling in speech recognition: The shared views of four research groups. *IEEE Signal Processing Magazine*, 29(6):82–97, 2012.
- Xuedong Huang and Li Deng. An overview of modern speech recognition., 2010.
- Xuedong Huang, Alex Acero, Hsiao-Wuen Hon, and Raj Foreword By-Reddy. *Spoken language processing: A guide to theory, algorithm, and system development*. Prentice hall PTR, 2001.
- Jack Kiefer, Jacob Wolfowitz, et al. Stochastic estimation of the maximum of a regression function. *The Annals of Mathematical Statistics*, 23(3):462–466, 1952.
- Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012.

- Aravindh Mahendran and Andrea Vedaldi. Understanding deep image representations by inverting them. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5188–5196, 2015.
- Grégoire Montavon, Sebastian Lapuschkin, Alexander Binder, Wojciech Samek, and Klaus-Robert Müller. Explaining nonlinear classification decisions with deep taylor decomposition. *Pattern Recognition*, 65:211–222, 2017.
- Douglas B Paul and Janet M Baker. The design for the wall street journal-based csr corpus. In *Proceedings of the workshop on Speech and Natural Language*, pages 357–362. Association for Computational Linguistics, 1992.
- Daniel Povey, Lukáš Burget, Mohit Agarwal, Pinar Akyazi, Kai Feng, Arnab Ghoshal, Ondřej Glembek, Nagendra Kumar Goel, Martin Karafiát, Ariya Rastrow, et al. Subspace gaussian mixture models for speech recognition. In *Acoustics Speech and Signal Processing (ICASSP), 2010 IEEE International Conference on*, pages 4330–4333. IEEE, 2010.
- Daniel Povey, Arnab Ghoshal, Gilles Boulianne, Lukas Burget, Ondrej Glembek, Nagendra Goel, Mirko Hannemann, Petr Motlicek, Yanmin Qian, Petr Schwarz, et al. The kaldi speech recognition toolkit. In *IEEE 2011 workshop on automatic speech recognition and understanding*, number EPFL-CONF-192584. IEEE Signal Processing Society, 2011.
- Sebastian Raschka. *Introduction to Artificial Neural Networks and Deep Learning*. Packt Publishing Ltd, 2017.
- Peter Mondrup Rasmussen, Tanya Schmäh, Kristoffer H Madsen, Torben E Lund, Grigori Yourganov, Stephen C Strother, and Lars Kai Hansen. Visualization of nonlinear classification models in neuroimaging-signed sensitivity maps. In *International Conference on Bio-inspired Systems and Signal Processing*, pages 254–263, 2012.

- Stuart Rosen and Peter Howell. *Signals and systems for speech and hearing*, volume 29. Brill, 2011.
- Frank Rosenblatt. The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological review*, 65(6):386, 1958.
- David E Rumelhart, Geoffrey E Hinton, Ronald J Williams, et al. Learning representations by back-propagating errors. *Cognitive modeling*, 5(3):1, 1988.
- Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.
- Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. Intriguing properties of neural networks. *arXiv preprint arXiv:1312.6199*, 2013.
- Olli Viikki and Kari Laurila. Cepstral domain segmental feature vector normalization for noise robust speech recognition. *Speech Communication*, 25(1):133–147, 1998.
- Jason Yosinski, Jeff Clune, Anh Nguyen, Thomas Fuchs, and Hod Lipson. Understanding neural networks through deep visualization. *arXiv preprint arXiv:1506.06579*, 2015.
- Steve Young. Hmms and related speech recognition technologies. In *Springer Handbook of Speech Processing*, pages 539–558. Springer, 2008.
- Matthew D Zeiler and Rob Fergus. Visualizing and understanding convolutional networks. In *European conference on computer vision*, pages 818–833. Springer, 2014.